

Додаток 1. Лістинг коду програми

apps/lab3.cpp

```
1 #include <AbstractSyntaxTree.hpp>
2 #include <Args.hpp>
3 #include <CodeGenerator.hpp>
4 #include <DeclarationsTableView.hpp>
5 #include <FileSource.hpp>
6 #include <IdentifiersTableView.hpp>
7 #include <LexicalError.hpp>
8 #include <LiteralTableView.hpp>
9 #include <Logger.hpp>
10 #include <Parser.hpp>
11 #include <SemanticAnalyzer.hpp>
12 #include <SemanticError.hpp>
13 #include <SymbolStore.hpp>
14 #include <SyntaxError.hpp>
15 #include <SyntaxTreeView.hpp>
16 #include <Tokenizer.hpp>
17 #include <TokensTableView.hpp>
18 #include <fstream>
19 #include <iostream>
20
21 int main(int argc, char* argv[]) {
22     Args args(argc, argv);
23     args.expectString("source", "s", true)
24         .expectString("output", "o", false)
25         .expectFlag("tokens", "t")
26         .expectFlag("tree", "T")
27         .expectFlag("identifiers", "i")
28         .expectFlag("literals", "l")
29         .expectFlag("declarations", "d")
30         .expectFlag("asm", "a")
31         .parse();
32
33     auto errorFormatter = [](const auto& err) {
34         return std::format(
35             "Error [{}:{}]: {}", err.row + 1, err.column + 1, err.message
36         );
37     };
38
39     SymbolStore symbols;
40     CharacterAttributes attributes;
41
42     FileSource source(args.getString("source"));
43     Logger<LexicalError> tokenizerLogger("Tokenizer", errorFormatter);
44     Tokenizer tokenizer(source, symbols, attributes, tokenizerLogger);
45     tokenizer.scan();
46
47     Logger<SyntaxError> parserLogger("Parser", errorFormatter);
48     Parser parser(symbols, tokenizer.tokens(), parserLogger);
49     parser.parse();
50
51     AbstractSyntaxTree ast(parser.tree(), symbols);
52
53     Logger<SemanticError> analyzerLogger("Semantics", errorFormatter);
54     SemanticAnalyzer semantics(ast, analyzerLogger);
55     semantics.analyze();
56
57     if (args.getFlag("tokens")) {
58         TokensTableView("\nTokens", symbols, tokenizer.tokens()).print();
59     }
60 }
```

```

61     if (args.getFlag("identifiers")) {
62         IdentifiersTableView("\nIdentifiers", symbols.identifiers()).print();
63     }
64
65     if (args.getFlag("literals")) {
66         LiteralsTableView("\nLiterals", symbols.literals()).print();
67     }
68
69     if (args.getFlag("tree")) {
70         SyntaxTreeView("\nSyntax Tree", parser.tree(), symbols).print();
71     }
72
73     if (args.getFlag("declarations")) {
74         DeclarationsTableView("\nDeclarations", semantics.declarations())
75             .print();
76     }
77
78     const bool hasErrors = !tokenizerLogger.messages().empty() ||
79                             !parserLogger.messages().empty() ||
80                             !analyzerLogger.messages().empty();
81
82     if (!hasErrors) {
83         AssemblyFormatter assemblyFormatter({
84             .labelWidth = 8,
85             .instructionWidth = 8,
86             .operandsWidth = 16,
87         });
88
89         CodeGenerator codegen(ast, assemblyFormatter);
90         codegen.generate();
91
92         const auto outputPath = args.getString("output");
93         if (!outputPath.empty()) {
94             std::ofstream file(outputPath);
95             file << codegen.output();
96             std::cout << "Assembly written to: " << outputPath << "\n";
97         }
98
99         if (args.getFlag("asm")) {
100             std::cout << "\n" << codegen.output();
101         }
102     }
103
104     return 0;
105 }

```

src/ast/data/AbstractSyntaxTree/AbstractSyntaxTree.hpp

```
1  #pragma once
2
3  #include <AstNode.hpp>
4  #include <Declaration.hpp>
5  #include <SymbolStore.hpp>
6  #include <SyntaxData.hpp>
7  #include <Tree.hpp>
8  #include <TreeNode.hpp>
9  #include <memory>
10
11 class AbstractSyntaxTree {
12 private:
13     const SymbolStore& _symbols;
14
15     void foldAxiom(const TreeNode<SyntaxData>& node);
16     std::unique_ptr<ProgramNode> foldProgram(const TreeNode<SyntaxData>& node);
17     void foldBlock(const TreeNode<SyntaxData>& node, ProgramNode& program);
18     void foldDeclarations(
19         const TreeNode<SyntaxData>& node,
20         ProgramNode& program
21     );
22     void foldConstantDeclarationsList(
23         const TreeNode<SyntaxData>& node,
24         ProgramNode& program
25     );
26     std::unique_ptr<ConstantNode> foldConstantDeclaration(
27         const TreeNode<SyntaxData>& node
28     );
29     Value evaluateConstant(const TreeNode<SyntaxData>& node);
30     Value evaluateComplexNumber(const TreeNode<SyntaxData>& node);
31
32 public:
33     std::unique_ptr<RootNode> root;
34
35     AbstractSyntaxTree(
36         const Tree<SyntaxData>& tree, const SymbolStore& symbols
37     );
38     void accept(class AstVisitor& visitor) const;
39 };
```

```
1  #include "AbstractSyntaxTree.hpp"
2
3  #include <AstNode.hpp>
4  #include <AstVisitor.hpp>
5  #include <Rules.hpp>
6  #include <complex>
7  #include <memory>
8  #include <optional>
9  #include <string>
10
11 AbstractSyntaxTree::AbstractSyntaxTree(
12     const Tree<SyntaxData>& tree,
13     const SymbolStore& symbols
14 ) :
15     _symbols(symbols), root(std::make_unique<RootNode>()) {
16     foldAxiom(tree.root());
17 }
18
19 void AbstractSyntaxTree::accept(AstVisitor& visitor) const {
20     root->accept(visitor);
21 }
22
23 void AbstractSyntaxTree::foldAxiom(const Tree<SyntaxData>& node) {
24     const auto& signalProgram = *node.children()[0];
25     const auto& program = *signalProgram.children()[0];
26
27     root->setProgram(foldProgram(program));
28 }
29
30 std::unique_ptr<ProgramNode> AbstractSyntaxTree::foldProgram(
31     const Tree<SyntaxData>& node
32 ) {
33     const auto& procedureIdentifier = *node.children()[0];
34     const auto& identifierTerminal = *procedureIdentifier.children()[0];
35     const auto& identifierToken = *identifierTerminal.data().token;
36
37     auto program = std::make_unique<ProgramNode>(
38         _symbols.lookup(identifierToken.code), identifierToken.row,
39         identifierToken.column
40     );
41
42     const auto& block = *node.children()[1];
43     foldBlock(block, *program);
44
45     return program;
46 }
47
48 void AbstractSyntaxTree::foldBlock(
49     const Tree<SyntaxData>& node,
50     ProgramNode& program
51 ) {
52     const auto& declarations = *node.children()[0];
53
54     foldDeclarations(declarations, program);
55     program.setStatements(std::make_unique<StatementsNode>());
56 }
57
58 void AbstractSyntaxTree::foldDeclarations(
59     const Tree<SyntaxData>& node,
60     ProgramNode& program
61 ) {
62     const auto& constantDeclarations = *node.children()[0];
```

```

63
64     if (constantDeclarations.data().rule ==
65         RuleKey::ConstantDeclarationsEmpty) {
66         return;
67     }
68
69     const auto& list = *constantDeclarations.children()[0];
70     foldConstantDeclarationsList(list, program);
71 }
72
73 void AbstractSyntaxTree::foldConstantDeclarationsList(
74     const TreeNode<SyntaxData>& node,
75     ProgramNode& program
76 ) {
77     if (node.data().rule == RuleKey::ConstantDeclarationsListEmpty) {
78         return;
79     }
80
81     const auto& declaration = *node.children()[0];
82     program.addConstant(foldConstantDeclaration(declaration));
83
84     const auto& rest = *node.children()[1];
85     foldConstantDeclarationsList(rest, program);
86 }
87
88 std::unique_ptr<ConstantNode> AbstractSyntaxTree::foldConstantDeclaration(
89     const TreeNode<SyntaxData>& node
90 ) {
91     const auto& constantIdentifier = *node.children()[0];
92     const auto& identifierTerminal = *constantIdentifier.children()[0];
93     const auto& identifierToken = *identifierTerminal.data().token;
94
95     const auto& constantValue = *node.children()[1];
96
97     return std::make_unique<ConstantNode>(
98         _symbols.lookup(identifierToken.code), evaluateConstant(constantValue),
99         identifierToken.row, identifierToken.column
100 );
101 }
102
103 Value AbstractSyntaxTree::evaluateConstant(const TreeNode<SyntaxData>& node) {
104     const auto& complexNumber = *node.children()[0];
105
106     return evaluateComplexNumber(complexNumber);
107 }
108
109 Value AbstractSyntaxTree::evaluateComplexNumber(
110     const TreeNode<SyntaxData>& node
111 ) {
112     const auto& leftPart = *node.children()[0];
113     const auto& rightPart = *node.children()[1];
114
115     std::optional<std::string> leftPartStr;
116     if (!leftPart.children().empty()) {
117         leftPartStr =
118             _symbols.lookup(leftPart.children()[0]->data().token->code);
119     }
120
121     std::optional<std::string> rightPartStr;
122     if (!rightPart.children().empty()) {
123         rightPartStr =
124             _symbols.lookup(rightPart.children()[0]->data().token->code);
125     }
126
127     if (rightPart.data().rule == RuleKey::RightPartExp) {
128         auto magnitude = std::stof(leftPartStr.value_or("1"));
129         auto exponent = std::stof(rightPartStr.value_or("0"));

```

```
130
131     return Value{std::polar(magnitude, exponent)};
132 }
133
134 auto real = std::stoi(leftPartStr.value_or("0"));
135 auto imaginary = std::stoi(rightPartStr.value_or("0"));
136
137 return Value{std::complex<int>{real, imaginary}};
138 }
```

src/ast/data/AstNode/AstNode.hpp

```
1  #pragma once
2
3  #include <AstVisitor.hpp>
4  #include <Declaration.hpp>
5  #include <cstdint>
6  #include <memory>
7  #include <string>
8  #include <vector>
9
10 class AstNode {
11 public:
12     virtual ~AstNode() = default;
13     virtual void accept(AstVisitor& visitor) const = 0;
14 };
15
16 class ConstantNode : public AstNode {
17 private:
18     std::string _identifier;
19     Value _value;
20     size_t _row;
21     size_t _column;
22
23 public:
24     ConstantNode(
25         std::string identifier, Value value, size_t row, size_t column
26     );
27
28     [[nodiscard]] const std::string& identifier() const;
29     [[nodiscard]] const Value& value() const;
30     [[nodiscard]] size_t row() const;
31     [[nodiscard]] size_t column() const;
32
33     void accept(AstVisitor& visitor) const override;
34 };
35
36 class StatementsNode : public AstNode {
37 public:
38     void accept(AstVisitor& visitor) const override;
39 };
40
41 class ProgramNode : public AstNode {
42 private:
43     std::string _identifier;
44     std::vector<std::unique_ptr<ConstantNode>> _constants;
45     std::unique_ptr<StatementsNode> _statements;
46     size_t _row;
47     size_t _column;
48
49 public:
50     ProgramNode(std::string identifier, size_t row, size_t column);
51
52     [[nodiscard]] const std::string& identifier() const;
53     [[nodiscard]] const std::vector<std::unique_ptr<ConstantNode>>& constants(
54     ) const;
55     [[nodiscard]] const StatementsNode* statements() const;
56     [[nodiscard]] size_t row() const;
57     [[nodiscard]] size_t column() const;
58
59     void addConstant(std::unique_ptr<ConstantNode> constant);
60     void setStatements(std::unique_ptr<StatementsNode> statements);
61
62     void accept(AstVisitor& visitor) const override;
```

```
63 };
64
65 class RootNode : public AstNode {
66 private:
67     std::unique_ptr<ProgramNode> _program;
68
69 public:
70     [[nodiscard]] const ProgramNode* program() const;
71
72     void setProgram(std::unique_ptr<ProgramNode> program);
73
74     void accept(AstVisitor& visitor) const override;
75 };
```



```
1  #include "AstNode.hpp"
2
3  #include <utility>
4
5  // --- ConstantNode ---
6
7  ConstantNode::ConstantNode(
8      std::string identifier,
9      Value value,
10     size_t row,
11     size_t column
12 ) :
13     _identifier(std::move(identifier)),
14     _value(std::move(value)),
15     _row(row),
16     _column(column) {}
17
18 const std::string& ConstantNode::identifier() const {
19     return _identifier;
20 }
21
22 const Value& ConstantNode::value() const {
23     return _value;
24 }
25
26 size_t ConstantNode::row() const {
27     return _row;
28 }
29
30 size_t ConstantNode::column() const {
31     return _column;
32 }
33
34 void ConstantNode::accept(AstVisitor& visitor) const {
35     visitor.visitConstantNode(*this);
36 }
37
38 // --- StatementsNode ---
39
40 void StatementsNode::accept(AstVisitor& visitor) const {
41     visitor.visitStatementsNode(*this);
42 }
43
44 // --- ProgramNode ---
45
46 ProgramNode::ProgramNode(std::string identifier, size_t row, size_t column) :
47     _identifier(std::move(identifier)), _row(row), _column(column) {}
48
49 const std::string& ProgramNode::identifier() const {
50     return _identifier;
51 }
52
53 const std::vector<std::unique_ptr<ConstantNode>>& ProgramNode::constants(
54 ) const {
55     return _constants;
56 }
57
58 const StatementsNode* ProgramNode::statements() const {
59     return _statements.get();
60 }
61
62 size_t ProgramNode::row() const {
```

```

63     return _row;
64 }
65
66 size_t ProgramNode::column() const {
67     return _column;
68 }
69
70 void ProgramNode::addConstant(std::unique_ptr<ConstantNode> constant) {
71     _constants.push_back(std::move(constant));
72 }
73
74 void ProgramNode::setStatements(std::unique_ptr<StatementsNode> statements) {
75     _statements = std::move(statements);
76 }
77
78 void ProgramNode::accept(AstVisitor& visitor) const {
79     visitor.visitProgramNode(*this);
80 }
81
82 // --- RootNode ---
83
84 const ProgramNode* RootNode::program() const {
85     return _program.get();
86 }
87
88 void RootNode::setProgram(std::unique_ptr<ProgramNode> program) {
89     _program = std::move(program);
90 }
91
92 void RootNode::accept(AstVisitor& visitor) const {
93     visitor.visitRootNode(*this);
94 }

```

src/ast/data/AstVisitor/AstVisitor.hpp

```
1 #pragma once
2
3 class ConstantNode;
4 class StatementsNode;
5 class ProgramNode;
6 class RootNode;
7
8 class AstVisitor {
9 public:
10     virtual ~AstVisitor() = default;
11
12     virtual void visitConstantNode(const ConstantNode& node) = 0;
13     virtual void visitStatementsNode(const StatementsNode& node) = 0;
14     virtual void visitProgramNode(const ProgramNode& node) = 0;
15     virtual void visitRootNode(const RootNode& node) = 0;
16 };
```

```
1 #pragma once
2
3 #include <string>
4
5 struct AssemblyFormatterConfig {
6     unsigned int labelWidth = 0;
7     unsigned int instructionWidth = 0;
8     unsigned int operandsWidth = 0;
9 };
10
11 class AssemblyFormatter {
12 private:
13     AssemblyFormatterConfig _config;
14
15     static std::string formatLabel(const std::string& label);
16     static std::string formatComment(const std::string& comment);
17
18     static std::string indent(unsigned int width);
19     static std::string pad(const std::string& text, unsigned int width);
20
21 public:
22     explicit AssemblyFormatter(AssemblyFormatterConfig config);
23
24     [[nodiscard]] std::string sectionLine(const std::string& name) const;
25     [[nodiscard]] std::string instructionLine(
26         const std::string& instruction
27     ) const;
28     [[nodiscard]] std::string instructionLine(
29         const std::string& instruction,
30         const std::string& operands
31     ) const;
32     [[nodiscard]] std::string constantLine(
33         const std::string& label,
34         const std::string& size,
35         const std::string& value
36     ) const;
37
38     [[nodiscard]] std::string commentLine(const std::string& comment) const;
39     [[nodiscard]] std::string labelLine(const std::string& label) const;
40     [[nodiscard]] std::string blankLine() const;
41 };
```

```
1 #include "AssemblyFormatter.hpp"
2
3 #include <format>
4
5 AssemblyFormatter::AssemblyFormatter(AssemblyFormatterConfig config) :
6     _config(config) {}
7
8 std::string AssemblyFormatter::commentLine(const std::string& comment) const {
9     return formatComment(comment) + "\n";
10 }
11
12 std::string AssemblyFormatter::blankLine() const {
13     return "\n";
14 }
15
16 std::string AssemblyFormatter::sectionLine(const std::string& name) const {
17     return indent(_config.labelWidth) + std::format("section {}: \n", name);
18 }
19
20 std::string AssemblyFormatter::labelLine(const std::string& label) const {
21     return formatLabel(label) + "\n";
22 }
23
24 std::string AssemblyFormatter::instructionLine(
25     const std::string& instruction
26 ) const {
27     return indent(_config.labelWidth) + instruction + "\n";
28 }
29
30 std::string AssemblyFormatter::instructionLine(
31     const std::string& instruction,
32     const std::string& operands
33 ) const {
34     auto instructionPad = _config.instructionWidth > 0 ? _config.instructionWidth - 1 : 0;
35
36     std::string line = indent(_config.labelWidth);
37     line += pad(instruction, instructionPad);
38     line += " ";
39     line += operands;
40     line += "\n";
41
42     return line;
43 }
44
45 std::string AssemblyFormatter::constantLine(
46     const std::string& label,
47     const std::string& size,
48     const std::string& value
49 ) const {
50     auto labelPad = _config.labelWidth > 0 ? _config.labelWidth - 1 : 0;
51     auto instructionPad = _config.instructionWidth > 0 ? _config.instructionWidth - 1 : 0;
52
53     std::string line = pad(formatLabel(label), labelPad);
54     line += " ";
55     line += pad(size, instructionPad);
56     line += " ";
57     line += value;
58     line += "\n";
59
60     return line;
61 }
62
```

```
63 std::string AssemblyFormatter::formatLabel(const std::string& label) {
64     return std::format("{}:", label);
65 }
66
67 std::string AssemblyFormatter::formatComment(const std::string& comment) {
68     return std::format("; {}", comment);
69 }
70
71 std::string AssemblyFormatter::indent(unsigned int width) {
72     return std::string(width, ' ');
73 }
74
75 std::string AssemblyFormatter::pad(
76     const std::string& text,
77     unsigned int width
78 ) {
79     if (text.length() >= width) {
80         return text;
81     }
82
83     return text + std::string(width - text.length(), ' ');
84 }
```

src/codegen/data/CodeGenerator/CodeGenerator.hpp

```
1 #pragma once
2
3 #include <AbstractSyntaxTree.hpp>
4 #include <AssemblyFormatter.hpp>
5 #include <AstNode.hpp>
6 #include <AstVisitor.hpp>
7 #include <sstream>
8 #include <string>
9
10 class CodeGenerator : public AstVisitor {
11 private:
12     const AbstractSyntaxTree& _ast;
13     const AssemblyFormatter& _formatter;
14     std::ostringstream _out;
15
16     void visitRootNode(const RootNode& node) override;
17     void visitProgramNode(const ProgramNode& node) override;
18     void visitConstantNode(const ConstantNode& node) override;
19     void visitStatementsNode(const StatementsNode& node) override;
20
21 public:
22     explicit CodeGenerator(
23         const AbstractSyntaxTree& ast,
24         const AssemblyFormatter& formatter
25     );
26
27     void generate();
28     [[nodiscard]] std::string output() const;
29 };
```

```
1  #include "CodeGenerator.hpp"
2
3  #include <complex>
4  #include <format>
5  #include <type_traits>
6  #include <variant>
7
8  CodeGenerator::CodeGenerator(
9      const AbstractSyntaxTree& ast,
10     const AssemblyFormatter& formatter
11 ) :
12     _ast(ast), _formatter(formatter) {}
13
14 void CodeGenerator::generate() {
15     _ast.accept(*this);
16 }
17
18 void CodeGenerator::visitRootNode(const RootNode& node) {
19     if (node.program()) {
20         node.program()->accept(*this);
21     }
22 }
23
24 void CodeGenerator::visitProgramNode(const ProgramNode& node) {
25     if (!node.constants().empty()) {
26         _out << _formatter.sectionLine(".data");
27
28         for (const auto& constant : node.constants()) {
29             constant->accept(*this);
30         }
31
32         _out << _formatter.blankLine();
33     }
34
35     _out << _formatter.sectionLine(".text");
36     _out << _formatter.instructionLine("global", "_start");
37
38     if (node.statements()) {
39         node.statements()->accept(*this);
40     }
41 }
42
43 void CodeGenerator::visitConstantNode(const ConstantNode& node) {
44     std::visit(
45         [&](const auto& value) {
46             using T = std::decay_t<decltype(value)>;
47             if constexpr (std::is_same_v<T, std::complex<int>>) {
48                 _out << _formatter.constantLine(
49                     node.identifier(), "dd",
50                     std::format("{}, {}", value.real(), value.imag())
51                 );
52             } else if constexpr (std::is_same_v<T, std::complex<float>>) {
53                 _out << _formatter.constantLine(
54                     node.identifier(), "dd",
55                     std::format("{:g}, {:g}", value.real(), value.imag())
56                 );
57             }
58         },
59         node.value()
60     );
61 }
62
```



```
63 void CodeGenerator::visitStatementsNode(const StatementsNode&) {
64     _out << _formatter.labelLine("_start");
65     _out << _formatter.instructionLine("nop");
66     _out << _formatter.instructionLine("mov", "eax, 60");
67     _out << _formatter.instructionLine("xor", "edi, edi");
68     _out << _formatter.instructionLine("syscall");
69 }
70
71 std::string CodeGenerator::output() const {
72     return _out.str();
73 }
```

src/declarations/data/Declaration/Declaration.hpp

```
1 #pragma once
2
3 #include <Type.hpp>
4 #include <complex>
5 #include <cstdint>
6 #include <optional>
7 #include <string>
8 #include <variant>
9
10 using Value = std::variant<std::complex<int>, std::complex<float>>;
11
12 enum class DeclarationKind {
13     Constant,
14     Program,
15 };
16
17 struct Declaration {
18     std::string identifier;
19     DeclarationKind kind;
20     std::optional<Type> type;
21     std::optional<Value> value;
22     size_t row = 0;
23     size_t column = 0;
24 };
```

src/declarations/data/DeclarationsTable/DeclarationsTable.hpp

```
1 #pragma once
2
3 #include <Declaration.hpp>
4 #include <optional>
5 #include <unordered_map>
6 #include <utility>
7 #include <vector>
8
9 class DeclarationsTable {
10 private:
11     std::unordered_map<std::string, Declaration> _declarations;
12
13 public:
14     [[nodiscard]] std::optional<Declaration> lookup(
15         const std::string& identifier
16     ) const;
17
18     std::pair<const Declaration*, bool> declare(Declaration declaration);
19
20     [[nodiscard]] std::vector<Declaration> entries() const;
21 };
```

```
1 #include "DeclarationsTable.hpp"
2
3 #include <algorithm>
4 #include <ranges>
5
6 std::optional<Declaration> DeclarationsTable::lookup(
7     const std::string& identifier
8 ) const {
9     if (!_declarations.contains(identifier)) {
10         return std::nullopt;
11     }
12
13     return _declarations.at(identifier);
14 }
15
16 std::pair<const Declaration*, bool> DeclarationsTable::declare(
17     Declaration declaration
18 ) {
19     auto [iterator, inserted] = _declarations.try_emplace(
20         declaration.identifier, std::move(declaration)
21     );
22
23     return {&iterator->second, inserted};
24 }
25
26 std::vector<Declaration> DeclarationsTable::entries() const {
27     auto result = std::ranges::to<std::vector<Declaration>>>(
28         std::views::values(_declarations)
29     );
30
31     std::ranges::sort(result, {}, &Declaration::identifier);
32
33     return result;
34 }
```

src/declarations/view/DeclarationsTableView/DeclarationsTableView.hpp

```
1 #pragma once
2
3 #include <Declaration.hpp>
4 #include <TableView.hpp>
5 #include <string>
6
7 #include "DeclarationsTable.hpp"
8
9 class DeclarationsTableView {
10 private:
11     TableView<Declaration> _table;
12
13 public:
14     DeclarationsTableView(std::string title, const DeclarationsTable& data);
15
16     void print() const;
17 };
```

src/declarations/view/DeclarationsTableView/DeclarationsTableView.cpp

```
1  #include "DeclarationsTableView.hpp"
2
3  #include <format>
4  #include <variant>
5
6  #include "Declaration.hpp"
7
8  namespace {
9  std::string extractDeclarationIdentifier(const Declaration& decl) {
10     return decl.identifier;
11 }
12
13 std::string extractDeclarationKind(const Declaration& decl) {
14     switch (decl.kind) {
15         case DeclarationKind::Program:
16             return "Program";
17         case DeclarationKind::Constant:
18             return "Constant";
19         default:
20             return "?";
21     }
22 }
23
24 std::string extractDeclarationType(const Declaration& decl) {
25     if (!decl.type) {
26         return "";
27     }
28
29     switch (decl.type.value()) {
30         case Type::ComplexInteger:
31             return "Complex Integer";
32         case Type::ComplexFloat:
33             return "Complex Float";
34         default:
35             return "?";
36     }
37 }
38
39 std::string extractDeclarationValue(const Declaration& decl) {
40     if (!decl.value) {
41         return "";
42     }
43
44     return std::visit(
45         [](const auto& item) -> std::string {
46             using T = std::decay_t<decltype(item)>;
47             if constexpr (std::is_same_v<T, std::complex<int>>) {
48                 return std::format("{} + i*{}", item.real(), item.imag());
49             } else {
50                 return std::format(
51                     "{:.3f} + i*{:.3f}", item.real(), item.imag()
52                 );
53             }
54         },
55         *decl.value
56     );
57 }
58
59 } // namespace
60
61 DeclarationsTableView::DeclarationsTableView(
62     std::string title,
```

```
63     const DeclarationsTable& data
64 ) :
65     _table(std::move(title), data.entries()) {
66     _table.addColumn("Identifier", "{:<20}", extractDeclarationIdentifier)
67         .addColumn("Kind", "{:<10}", extractDeclarationKind)
68         .addColumn("Type", "{:<20}", extractDeclarationType)
69         .addColumn("Value", "{:<30}", extractDeclarationValue);
70 }
71
72 void DeclarationsTableView::print() const {
73     _table.print();
74 }
```

```
1 #pragma once
2
3 #include <string>
4 #include <unordered_map>
5
6 struct Rule {
7     std::string id;
8     std::string input;
9     std::string output;
10 };
11
12 enum class RuleKey {
13     Axiom,
14     SignalProgram,
15     Program,
16     Block,
17     StatementsList,
18     Declarations,
19     ConstantDeclarations,
20     ConstantDeclarationsEmpty,
21     ConstantDeclarationsList,
22     ConstantDeclarationsListEmpty,
23     ConstantDeclaration,
24     Constant,
25     ComplexNumber,
26     LeftPartValue,
27     LeftPartEmpty,
28     RightPartBase,
29     RightPartExp,
30     RightPartEmpty,
31     ConstantIdentifier,
32     ProcedureIdentifier,
33     Terminal,
34 };
35
36 inline const std::unordered_map<RuleKey, Rule> RULES = {
37     {RuleKey::Axiom, {.id = "S", .input = "", .output = "<signal-program>"}},
38     {RuleKey::SignalProgram,
39      {.id = "1", .input = "<signal-program>", .output = "<program>"}},
40     {RuleKey::Program,
41      {.id = "2",
42       .input = "<program>",
43       .output = "PROGRAM <procedure-identifier>; <block>."}},
44     {RuleKey::Block,
45      {.id = "3",
46       .input = "<block>",
47       .output = "<declarations> BEGIN <statements-list> END"}},
48     {RuleKey::StatementsList,
49      {.id = "4", .input = "<statements-list>", .output = "<empty>"}},
50     {RuleKey::Declarations,
51      {.id = "5",
52       .input = "<declarations>",
53       .output = "<constant-declarations>"}},
54     {RuleKey::ConstantDeclarations,
55      {.id = "6",
56       .input = "<constant-declarations>",
57       .output = "CONST <constant-declarations-list>"}},
58     {RuleKey::ConstantDeclarationsEmpty,
59      {.id = "6", .input = "<constant-declarations>", .output = "<empty>"}},
60     {RuleKey::ConstantDeclarationsList,
61      {.id = "7",
62       .input = "<constant-declarations-list>",
```



```

63     .output = "<constant-declaration> <constant-declarations-list>"}},
64 {RuleKey::ConstantDeclarationsListEmpty,
65  {id = "7", .input = "<constant-declarations-list>", .output = "<empty>"}},
66 {RuleKey::ConstantDeclaration,
67  {id = "8",
68   .input = "<constant-declaration>",
69   .output = "<constant-identifier> = <constant>;"}},
70 {RuleKey::Constant,
71  {id = "9", .input = "<constant>", .output = "'<complex-number>'"}},
72 {RuleKey::ComplexNumber,
73  {id = "10",
74   .input = "<complex-number>",
75   .output = "<left-part> <right-part>"}},
76 {RuleKey::LeftPartValue,
77  {id = "11", .input = "<left-part>", .output = "<unsigned-integer>"}},
78 {RuleKey::LeftPartEmpty,
79  {id = "11", .input = "<left-part>", .output = "<empty>"}},
80 {RuleKey::RightPartBase,
81  {id = "12", .input = "<right-part>", .output = ", <unsigned-integer>"}},
82 {RuleKey::RightPartExp,
83  {id = "12",
84   .input = "<right-part>",
85   .output = "$EXP( <unsigned-integer> )"}},
86 {RuleKey::RightPartEmpty,
87  {id = "12", .input = "<right-part>", .output = "<empty>"}},
88 {RuleKey::ConstantIdentifier,
89  {id = "13", .input = "<constant-identifier>", .output = "<identifier>"}},
90 {RuleKey::ProcedureIdentifier,
91  {id = "14", .input = "<procedure-identifier>", .output = "<identifier>"}},
92 };

```

```
1 #pragma once
2
3 #include <Logger.hpp>
4 #include <ParsingScope.hpp>
5 #include <Queue.hpp>
6 #include <Stack.hpp>
7 #include <SymbolStore.hpp>
8 #include <SyntaxData.hpp>
9 #include <SyntaxError.hpp>
10 #include <Tokenizer.hpp>
11 #include <Tree.hpp>
12
13 class Parser {
14 private:
15     const SymbolStore& _symbols;
16     Logger<SyntaxError>& _logger;
17
18     Queue<Token> _tokens;
19
20     Tree<SyntaxData> _tree;
21     Stack<TreeNode<SyntaxData>*> _nodes;
22
23     // 1. <signal-program> --> <program>
24     void parseSignalProgram();
25     // 2. <program> --> PROGRAM <procedure-identifier>; <block>.
26     void parseProgram();
27     // 3. <block> --> <declarations> BEGIN <statements-list> END
28     void parseBlock();
29     // 4. <statements-list> --> <empty>
30     void parseStatementsList();
31     // 5. <declarations> --> <constant-declarations>
32     void parseDeclarations();
33     // 6. <constant-declarations> --> CONST <constant-declarations-list> |
34     // <empty>
35     void parseConstantDeclarations();
36     // 7. <constant-declarations-list> --> <constant-declaration>
37     // <constant-declarations-list> | <empty>
38     void parseConstantDeclarationsList();
39     // 8. <constant-declaration> --> <constant-identifier> = <constant>;
40     void parseConstantDeclaration();
41     // 9. <constant> --> '<complex-number>'
42     void parseConstant();
43     // 10. <complex-number> --> <left-part> <right-part>
44     void parseComplexNumber();
45     // 11. <left-part> --> <unsigned-integer> | <empty>
46     void parseLeftPart();
47     // 12. <right-part> --> ,<unsigned-integer> | $EXP( <unsigned-integer> ) |
48     // <empty>
49     void parseRightPart();
50     // 13. <constant-identifier> --> <identifier>
51     void parseConstantIdentifier();
52     // 14. <procedure-identifier> --> <identifier>
53     void parseProcedureIdentifier();
54     // 15. <identifier> --> <letter><string>
55     void parseIdentifier(const std::string& errorMessage);
56     // 17. <unsigned-integer> --> <digit><digits-string>
57     void parseUnsignedInteger();
58
59     void skipToProcedureBody();
60     void skipToBlockEnd();
61     void skipToNextDeclaration();
62
```

```

63     Token expect(const std::string& expected, const std::string& errorMessage);
64     Token expect(SymbolType expected, const std::string& errorMessage);
65
66     bool consider(const std::string& expected);
67     bool consider(SymbolType expected);
68
69     [[noreturn]] void fail(const std::string& message);
70     [[noreturn]] void fail(const std::string& message, const Token& token);
71
72     [[nodiscard]] ParsingScope grow(const SyntaxData& data);
73
74 public:
75     Parser(
76         const SymbolStore& symbols,
77         const std::vector<Token>& tokens,
78         Logger<SyntaxError>& logger
79     );
80
81     void parse();
82     const Tree<SyntaxData>& tree() const;
83 };

```

```
1 #include "Parser.hpp"
2
3 #include <Rules.hpp>
4
5 Parser::Parser(
6     const SymbolStore& symbols,
7     const std::vector<Token>& tokens,
8     Logger<SyntaxError>& logger
9 ) :
10     _symbols(symbols),
11     _logger(logger),
12     _tokens(tokens),
13     _tree(Tree<SyntaxData>({.rule = RuleKey::Axiom, .token = _tokens.peek()})) {
14     _nodes.push(&_tree.root());
15 }
16
17 void Parser::parse() {
18     parseSignalProgram();
19 }
20
21 // 1. <signal-program> --> <program>
22 void Parser::parseSignalProgram() {
23     auto scope =
24         grow({.rule = RuleKey::SignalProgram, .token = _tokens.peek()});
25
26     try {
27         parseProgram();
28     } catch (const SyntaxError&) {
29         // If parsing fails at the top level, we can't really recover,
30         // so we just clear the remaining tokens to prevent cascading errors.
31         _tokens.clear();
32     }
33
34     if (!_tokens.isEmpty()) {
35         auto token = _tokens.peek();
36
37         _logger.message({
38             .message = SyntaxError::UnexpectedSymbolsAfterEndOfProgram,
39             .row = token->row,
40             .column = token->column,
41         });
42     }
43 }
44
45 // 2. <program> --> PROGRAM <procedure-identifier>; <block>.
46 void Parser::parseProgram() {
47     auto scope = grow({.rule = RuleKey::Program, .token = _tokens.peek()});
48
49     try {
50         expect(Keyword::Program, SyntaxError::MustStartWithProgram);
51         parseProcedureIdentifier();
52         expect(Delimiter::Semicolon, SyntaxError::ExpectedSemicolon);
53     } catch (const SyntaxError&) {
54         skipToProcedureBody();
55     }
56
57     try {
58         parseBlock();
59     } catch (const SyntaxError&) {
60         skipToBlockEnd();
61     }
62 }
```

```

63     expect(Delimiter::Dot, SyntaxError::ExpectedDot);
64 }
65
66 // 3. <block> --> <declarations> BEGIN <statements-list> END
67 void Parser::parseBlock() {
68     auto scope = grow({.rule = RuleKey::Block, .token = _tokens.peek()});
69
70     parseDeclarations();
71     expect(Keyword::Begin, SyntaxError::ExpectedBeginKeyword);
72     parseStatementsList();
73     expect(Keyword::End, SyntaxError::ExpectedEndKeyword);
74 }
75
76 // 4. <statements-list> --> <empty>
77 void Parser::parseStatementsList() {
78     auto scope =
79         grow({.rule = RuleKey::StatementsList, .token = _tokens.peek()});
80 }
81
82 // 5. <declarations> --> <constant-declarations>
83 void Parser::parseDeclarations() {
84     auto scope = grow({.rule = RuleKey::Declarations, .token = _tokens.peek()});
85
86     parseConstantDeclarations();
87 }
88
89 // 6. <constant-declarations> --> CONST <constant-declarations-list> | <empty>
90 void Parser::parseConstantDeclarations() {
91     if (consider(Keyword::Const)) {
92         auto scope = grow(
93             {.rule = RuleKey::ConstantDeclarations, .token = _tokens.peek()}
94         );
95
96         expect(Keyword::Const, SyntaxError::ExpectedConstKeyword);
97         parseConstantDeclarationsList();
98     } else {
99         auto scope = grow(
100             {.rule = RuleKey::ConstantDeclarationsEmpty,
101              .token = _tokens.peek()}
102         );
103     }
104 }
105
106 // 7. <constant-declarations-list> --> <constant-declaration>
107 // <constant-declarations-list> | <empty>
108 void Parser::parseConstantDeclarationsList() {
109     if (!consider(Keyword::Begin)) {
110         auto scope = grow(
111             {.rule = RuleKey::ConstantDeclarationsList, .token = _tokens.peek()}
112         );
113
114         try {
115             parseConstantDeclaration();
116         } catch (const SyntaxError&) {
117             skipToNextDeclaration();
118         }
119
120         parseConstantDeclarationsList();
121     } else {
122         auto scope = grow(
123             {.rule = RuleKey::ConstantDeclarationsListEmpty,
124              .token = _tokens.peek()}
125         );
126     }
127 }
128
129 // 8. <constant-declaration> --> <constant-identifier> = <constant>;

```

```

130 void Parser::parseConstantDeclaration() {
131     auto scope =
132         grow({.rule = RuleKey::ConstantDeclaration, .token = _tokens.peek()});
133
134     parseConstantIdentifier();
135     expect(Delimiter::Equals, SyntaxError::ExpectedEquals);
136     parseConstant();
137     try {
138         expect(Delimiter::Semicolon, SyntaxError::ExpectedConstantSemicolon);
139     } catch (const SyntaxError&) {
140         // Implicitly add a semicolon to recover and continue parsing the next
141         // declarations.
142         // TODO: check that this does not backfire for severely mangled
143         // declarations.
144     }
145 }
146
147 // 9. <constant> --> '<complex-number>'
148 void Parser::parseConstant() {
149     auto scope = grow({.rule = RuleKey::Constant, .token = _tokens.peek()});
150
151     expect(Delimiter::Quote, SyntaxError::ExpectedOpeningQuote);
152     parseComplexNumber();
153     expect(Delimiter::Quote, SyntaxError::ExpectedClosingQuote);
154 }
155
156 // 10. <complex-number> --> <left-part> <right-part>
157 void Parser::parseComplexNumber() {
158     auto scope =
159         grow({.rule = RuleKey::ComplexNumber, .token = _tokens.peek()});
160
161     parseLeftPart();
162     parseRightPart();
163 }
164
165 // 11. <left-part> --> <unsigned-integer> | <empty>
166 void Parser::parseLeftPart() {
167     if (consider(SymbolType::Literal)) {
168         auto scope =
169             grow({.rule = RuleKey::LeftPartValue, .token = _tokens.peek()});
170
171         parseUnsignedInteger();
172     } else {
173         auto scope =
174             grow({.rule = RuleKey::LeftPartEmpty, .token = _tokens.peek()});
175     }
176 }
177
178 // 12. <right-part> --> ,<unsigned-integer> | $EXP( <unsigned-integer> ) |
179 // <empty>
180 void Parser::parseRightPart() {
181     if (consider(Delimiter::Comma)) {
182         auto scope =
183             grow({.rule = RuleKey::RightPartBase, .token = _tokens.peek()});
184
185         expect(Delimiter::Comma, SyntaxError::ExpectedComma);
186         parseUnsignedInteger();
187         return;
188     }
189
190     if (consider(MultiDelimiter::Exp)) {
191         auto scope =
192             grow({.rule = RuleKey::RightPartExp, .token = _tokens.peek()});
193
194         expect(MultiDelimiter::Exp, SyntaxError::ExpectedExp);
195         expect(Delimiter::OpenParenthesis, SyntaxError::ExpectedOpenParen);
196         parseUnsignedInteger();

```

```

197     expect(Delimiter::CloseParenthesis, SyntaxError::ExpectedCloseParen);
198     return;
199 }
200
201 auto scope =
202     grow({.rule = RuleKey::RightPartEmpty, .token = _tokens.peek()});
203 }
204
205 // 13. <constant-identifier> --> <identifier>
206 void Parser::parseConstantIdentifier() {
207     auto scope =
208         grow({.rule = RuleKey::ConstantIdentifier, .token = _tokens.peek()});
209
210     parseIdentifier(SyntaxError::ExpectedConstantIdentifier);
211 }
212
213 // 14. <procedure-identifier> --> <identifier>
214 void Parser::parseProcedureIdentifier() {
215     auto scope =
216         grow({.rule = RuleKey::ProcedureIdentifier, .token = _tokens.peek()});
217
218     parseIdentifier(SyntaxError::ExpectedProcedureIdentifier);
219 }
220
221 // 15. <identifier> --> <letter><string> (terminal)
222 void Parser::parseIdentifier(const std::string& errorMessage) {
223     auto token = expect(SymbolType::Identifier, errorMessage);
224
225     auto scope = grow({.rule = RuleKey::Terminal, .token = token});
226 }
227
228 // 17. <unsigned-integer> --> <digit><digits-string> (terminal)
229 void Parser::parseUnsignedInteger() {
230     auto token =
231         expect(SymbolType::Literal, SyntaxError::ExpectedUnsignedInteger);
232
233     auto scope = grow({.rule = RuleKey::Terminal, .token = token});
234 }
235
236 // --- Error recovery ---
237
238 void Parser::skipToProcedureBody() {
239     auto found = _tokens.popUntil([&](const Token& token) {
240         const auto& symbol = _symbols.lookup(token.code);
241         return symbol == Keyword::Const || symbol == Keyword::Begin;
242     });
243
244     if (!found) {
245         fail(SyntaxError::UnexpectedEndOfFile);
246     }
247 }
248
249 void Parser::skipToBlockEnd() {
250     auto found = _tokens.popUntil([&](const Token& token) {
251         const auto& symbol = _symbols.lookup(token.code);
252         return symbol == Keyword::End || symbol == Delimiter::Dot;
253     });
254
255     if (!found) {
256         fail(SyntaxError::UnexpectedEndOfFile);
257     }
258
259     if (_symbols.lookup(found->code) == Keyword::End) {
260         _tokens.pop();
261     }
262 }
263

```

```

264 void Parser::skipToNextDeclaration() {
265     auto found = _tokens.popUntil([&](const Token& token) {
266         const auto& symbol = _symbols.lookup(token.code);
267         return symbol == Delimiter::Semicolon || symbol == Keyword::Begin;
268     });
269
270     if (!found) {
271         fail(SyntaxError::UnexpectedEndOfFile);
272     }
273
274     if (_symbols.lookup(found->code) == Delimiter::Semicolon) {
275         _tokens.pop();
276     }
277 }
278
279 // --- Utilities ---
280
281 Token Parser::expect(
282     const std::string& expected,
283     const std::string& errorMessage
284 ) {
285     auto token = _tokens.peek();
286
287     if (!token) {
288         fail(errorMessage);
289     }
290
291     if (_symbols.lookup(token->code) != expected) {
292         fail(errorMessage, *token);
293     }
294
295     return *_tokens.pop();
296 }
297
298 Token Parser::expect(SymbolType expected, const std::string& errorMessage) {
299     auto token = _tokens.peek();
300
301     if (!token) {
302         fail(errorMessage);
303     }
304
305     if (_symbols.lookupType(token->code) != expected) {
306         fail(errorMessage, *token);
307     }
308
309     return *_tokens.pop();
310 }
311
312 bool Parser::consider(const std::string& expected) {
313     auto token = _tokens.peek();
314
315     return token && _symbols.lookup(token->code) == expected;
316 }
317
318 bool Parser::consider(SymbolType expected) {
319     auto token = _tokens.peek();
320
321     return token && _symbols.lookupType(token->code) == expected;
322 }
323
324 void Parser::fail(const std::string& message) {
325     SyntaxError error{.message = message, .row = 0, .column = 0};
326
327     _logger.message(error);
328
329     throw error;
330 }

```



```

331
332 void Parser::fail(const std::string& message, const Token& token) {
333     SyntaxError error{
334         .message = message, .row = token.row, .column = token.column
335     };
336
337     _logger.message(error);
338
339     throw error;
340 }
341
342 ParsingScope Parser::grow(const SyntaxData& data) {
343     auto parentNode = _nodes.peek();
344     auto& node = (*parentNode)->grow(data);
345     _nodes.push(&node);
346     return ParsingScope(_nodes);
347 }
348
349 const Tree<SyntaxData>& Parser::tree() const {
350     return _tree;
351 }

```

src/parser/data/ParsingScope/ParsingScope.hpp

```
1 #pragma once
2
3 #include <Stack.hpp>
4 #include <SyntaxData.hpp>
5 #include <Tree.hpp>
6
7 class ParsingScope {
8     Stack<TreeNode<SyntaxData>*>* _stack;
9
10 public:
11     explicit ParsingScope(Stack<TreeNode<SyntaxData>*>& stack);
12     ~ParsingScope();
13
14     ParsingScope(ParsingScope&& other) noexcept;
15     ParsingScope(const ParsingScope&) = delete;
16     ParsingScope& operator=(const ParsingScope&) = delete;
17     ParsingScope& operator=(ParsingScope&&) = delete;
18 };
```

src/parser/data/ParsingScope/ParsingScope.cpp

```
1 #include "ParsingScope.hpp"
2
3 ParsingScope::ParsingScope(Stack<TreeNode<SyntaxData>*>& stack) :
4     _stack(&stack) {}
5
6 ParsingScope::~ParsingScope() {
7     if (_stack) {
8         _stack->pop();
9     }
10 }
11
12 ParsingScope::ParsingScope(ParsingScope&& other) noexcept :
13     _stack(other._stack) {
14     other._stack = nullptr;
15 }
```

src/parser/data/SyntaxData/SyntaxData.hpp

```
1 #pragma once
2
3 #include <optional>
4
5 #include <Rules.hpp>
6 #include <Tokenizer.hpp>
7
8 struct SyntaxData {
9     RuleKey rule;
10     std::optional<Token> token;
11 };
```

```
1 #pragma once
2
3 #include <cstdint>
4 #include <string>
5
6 struct SyntaxError {
7     std::string message;
8     size_t row;
9     size_t column;
10
11     // Error message constants
12     static constexpr auto MustStartWithProgram =
13         "Program must start with 'PROGRAM' keyword";
14     static constexpr auto ExpectedProcedureIdentifier =
15         "Expected procedure identifier after 'PROGRAM' keyword";
16     static constexpr auto ExpectedSemicolon =
17         "Expected ';' after procedure identifier";
18     static constexpr auto ExpectedDot =
19         "Expected '.' at the end of the program";
20     static constexpr auto ExpectedBeginKeyword =
21         "Expected 'BEGIN' keyword before statements list";
22     static constexpr auto ExpectedEndKeyword =
23         "Expected 'END' keyword at the end of the block";
24     static constexpr auto ExpectedConstKeyword =
25         "Expected 'CONST' keyword before constant declarations";
26     static constexpr auto ExpectedIdentifier = "Expected identifier";
27     static constexpr auto ExpectedConstantIdentifier =
28         "Expected constant identifier in constant declaration";
29     static constexpr auto ExpectedEquals =
30         "Expected '=' in constant declaration";
31     static constexpr auto ExpectedConstantSemicolon =
32         "Expected ';' at the end of constant declaration";
33     static constexpr auto ExpectedOpeningQuote =
34         "Expected '"' (single quote) at the beginning of constant declaration";
35     static constexpr auto ExpectedClosingQuote =
36         "Expected '"' (single quote) at the end of constant declaration";
37     static constexpr auto ExpectedUnsignedInteger =
38         "Expected unsigned integer literal";
39     static constexpr auto ExpectedComma = "Expected ',' in complex number";
40     static constexpr auto ExpectedExp = "Expected '$EXP' in complex number";
41     static constexpr auto ExpectedOpenParen =
42         "Expected '(' after '$EXP' in complex number";
43     static constexpr auto ExpectedCloseParen =
44         "Expected ')' after exponent in complex number";
45     static constexpr auto UnexpectedEndOfFile = "Unexpected end of file";
46     static constexpr auto UnexpectedSymbolsAfterEndOfProgram =
47         "Unexpected symbols after end of program";
48 };
```

src/parser/view/DotTreeView/DotTreeView.hpp

```
1 #pragma once
2
3 #include <string>
4
5 #include <Parser.hpp>
6 #include <SymbolStore.hpp>
7
8 class DotTreeView {
9 private:
10     const Tree<SyntaxData>& _tree;
11     const SymbolStore& _symbols;
12
13     std::string nodeLabel(const SyntaxData& data) const;
14     std::string edgeLabel(const SyntaxData& data) const;
15
16     void writeNode(
17         std::ostream& out,
18         const TreeNode<SyntaxData>& node,
19         int& id
20     ) const;
21
22 public:
23     DotTreeView(const Tree<SyntaxData>& tree, const SymbolStore& symbols);
24
25     void write(const std::string& filename) const;
26 };
```

src/parser/view/DotTreeView/DotTreeView.cpp

```
1  #include "DotTreeView.hpp"
2
3  #include <fstream>
4
5  #include <Rules.hpp>
6
7  DotTreeView::DotTreeView(
8      const Tree<SyntaxData>& tree,
9      const SymbolStore& symbols
10 ) :
11     _tree(tree), _symbols(symbols) {}
12
13 std::string DotTreeView::nodeLabel(const SyntaxData& data) const {
14     if (data.rule == RuleKey::Terminal) {
15         return _symbols.lookup(data.token->code);
16     }
17     return RULES.at(data.rule).output;
18 }
19
20 std::string DotTreeView::edgeLabel(const SyntaxData& data) const {
21     if (data.rule == RuleKey::Terminal) {
22         return "T";
23     }
24     return RULES.at(data.rule).id;
25 }
26
27 void DotTreeView::writeNode(
28     std::ostream& out,
29     const TreeNode<SyntaxData>& node,
30     int& id
31 ) const {
32     int nodeId = id++;
33     const auto& data = node.data();
34
35     out << "    n" << nodeId << " [label=\"" << nodeLabel(data) << "\"]\n";
36
37     for (const auto& child : node.children()) {
38         int childId = id;
39         writeNode(out, *child, id);
40
41         int ruleId = id++;
42         out << "    r" << ruleId << " [label=\"" << edgeLabel(child->data())
43             << "\" shape=circle width=0.3 fixedsize=true"
44             << " fontsize=10]\n";
45
46         out << "    n" << nodeId << " -> r" << ruleId
47             << " [arrowhead=none]\n";
48         out << "    r" << ruleId << " -> n" << childId << "\n";
49     }
50 }
51
52 void DotTreeView::write(const std::string& filename) const {
53     std::ofstream out(filename);
54
55     out << "digraph SyntaxTree {\n"
56         << "    rankdir=TB\n"
57         << "    node [shape=none fontname=\"Consolas\" fontsize=11]\n"
58         << "    edge [arrowsize=0.6]\n\n";
59
60     int id = 0;
61     writeNode(out, _tree.root(), id);
62 }
```

```
63     out << "}\n";
64 }
```


src/parser/view/SyntaxTreeView/SyntaxTreeView.hpp

```
1 #pragma once
2
3 #include <string>
4
5 #include <Parser.hpp>
6 #include <SymbolStore.hpp>
7 #include <TreeView.hpp>
8
9 class SyntaxTreeView {
10 private:
11     TreeView<SyntaxData> _tree;
12
13 public:
14     SyntaxTreeView(
15         std::string title,
16         const Tree<SyntaxData>& tree,
17         const SymbolStore& symbols
18     );
19
20     void print() const;
21 };
```

```
1 #include "SyntaxTreeView.hpp"
2
3 #include <format>
4
5 #include <Rules.hpp>
6
7 SyntaxTreeView::SyntaxTreeView(
8     std::string title,
9     const Tree<SyntaxData>& tree,
10    const SymbolStore& symbols
11 ) :
12     _tree(std::move(title), tree) {
13     _tree
14         .setNodeFormatter([&symbols](const auto& data) {
15             if (data.rule == RuleKey::Terminal) {
16                 return symbols.lookup(data.token->code);
17             }
18             return RULES.at(data.rule).output;
19         })
20         .setEdgeFormatter([](const auto& data) {
21             if (data.rule == RuleKey::Terminal) {
22                 return std::string("(T)");
23             }
24             return std::format("({})", RULES.at(data.rule).id);
25         });
26 }
27
28 void SyntaxTreeView::print() const {
29     _tree.print();
30 }
```

```
1 #pragma once
2
3 #include <AbstractSyntaxTree.hpp>
4 #include <AstVisitor.hpp>
5 #include <Declaration.hpp>
6 #include <DeclarationsTable.hpp>
7 #include <Logger.hpp>
8 #include <SemanticError.hpp>
9
10 class SemanticAnalyzer : public AstVisitor {
11 private:
12     const AbstractSyntaxTree& _ast;
13     Logger<SemanticError>& _logger;
14
15     DeclarationsTable _declarations;
16
17 public:
18     SemanticAnalyzer(
19         const AbstractSyntaxTree& ast, Logger<SemanticError>& logger
20     );
21
22     void analyze();
23     [[nodiscard]] const DeclarationsTable& declarations() const;
24
25     void visitRootNode(const RootNode& node) override;
26     void visitProgramNode(const ProgramNode& node) override;
27     void visitConstantNode(const ConstantNode& node) override;
28     void visitStatementsNode(const StatementsNode& node) override;
29 };
```

```
1  #include "SemanticAnalyzer.hpp"
2
3  #include <AstNode.hpp>
4  #include <Declaration.hpp>
5  #include <SemanticError.hpp>
6  #include <complex>
7  #include <variant>
8
9  namespace {
10
11  Type typeOf(const Value& value) {
12      return std::visit(
13          [](const auto& alternative) {
14              using T = std::decay_t<decltype(alternative)>;
15              if constexpr (std::is_same_v<T, std::complex<float>>) {
16                  return Type::ComplexFloat;
17              } else {
18                  return Type::ComplexInteger;
19              }
20          },
21          value
22      );
23  }
24
25  } // namespace
26
27  SemanticAnalyzer::SemanticAnalyzer(
28      const AbstractSyntaxTree& ast,
29      Logger<SemanticError>& logger
30  ) :
31      _ast(ast), _logger(logger) {}
32
33  void SemanticAnalyzer::analyze() {
34      _ast.accept(*this);
35  }
36
37  const DeclarationsTable& SemanticAnalyzer::declarations() const {
38      return _declarations;
39  }
40
41  void SemanticAnalyzer::visitRootNode(const RootNode& node) {
42      if (node.program()) {
43          node.program()->accept(*this);
44      }
45  }
46
47  void SemanticAnalyzer::visitProgramNode(const ProgramNode& node) {
48      auto [existing, inserted] = _declarations.declare({
49          .identifier = node.identifier(),
50          .kind = DeclarationKind::Program,
51          .type = std::nullopt,
52          .value = std::nullopt,
53          .row = node.row(),
54          .column = node.column(),
55      });
56
57      if (!inserted) {
58          _logger.message({
59              .message = SemanticError::DuplicateIdentifier(
60                  node.identifier(), existing->row, existing->column
61              ),
62              .row = node.row(),
```

```

63         .column = node.column(),
64     });
65 }
66
67     for (const auto& constant : node.constants()) {
68         constant->accept(*this);
69     }
70 }
71
72 void SemanticAnalyzer::visitStatementsNode(const StatementsNode& node) {}
73
74 void SemanticAnalyzer::visitConstantNode(const ConstantNode& node) {
75     auto [existing, inserted] = _declarations.declare({
76         .identifier = node.identifier(),
77         .kind = DeclarationKind::Constant,
78         .type = typeOf(node.value()),
79         .value = node.value(),
80         .row = node.row(),
81         .column = node.column(),
82     });
83
84     if (!inserted) {
85         _logger.message({
86             .message = SemanticError::DuplicateIdentifier(
87                 node.identifier(), existing->row, existing->column
88             ),
89             .row = node.row(),
90             .column = node.column(),
91         });
92     }
93 }

```

```
1 #pragma once
2
3 #include <cstdint>
4 #include <format>
5 #include <string>
6
7 struct SemanticError {
8     std::string message;
9     size_t row;
10    size_t column;
11
12    // Error message constants
13    static auto DuplicateIdentifier(
14        std::string identifier,
15        size_t originalRow,
16        size_t originalColumn
17    ) {
18        return std::format(
19            "Identifier '{}' is already declared at [{}:{}]", identifier,
20            originalRow + 1, originalColumn + 1
21        );
22    };
23 };
```

src/symbols/symbols.constants.hpp

```
1 #pragma once
2
3 #include <cstdint>
4
5 constexpr size_t ASCII_TABLE_SIZE = 256;
6
7 namespace Keyword {
8     constexpr auto Program = "PROGRAM";
9     constexpr auto Const = "CONST";
10    constexpr auto Begin = "BEGIN";
11    constexpr auto End = "END";
12 } // namespace Keyword
13
14 namespace MultiDelimiter {
15     constexpr auto Exp = "$EXP";
16 }
17
18 namespace Delimiter {
19     constexpr auto Semicolon = ";";
20     constexpr auto Dot = ".";
21     constexpr auto Equals = "=";
22     constexpr auto Quote = "'";
23     constexpr auto Comma = ",";
24     constexpr auto OpenParenthesis = "(";
25     constexpr auto CloseParenthesis = ")";
26 } // namespace Delimiter
```

```
1  #pragma once
2
3  #include <array>
4  #include <string>
5  #include <unordered_map>
6  #include <vector>
7
8  #include "symbols.constants.hpp"
9
10 enum class SymbolType {
11     Ascii,
12     MultiDelimiter,
13     Keyword,
14     Literal,
15     Identifier,
16 };
17
18 const size_t DELIMITERS_OFFSET = 0;
19 const size_t MULTI_DELIMITERS_OFFSET = ASCII_TABLE_SIZE;
20 const size_t KEYWORDS_OFFSET = 300;
21 const size_t LITERALS_OFFSET = 500;
22 const size_t IDENTIFIERS_OFFSET = 1000;
23 const size_t MAX_TOKENS = 2000;
24
25 class SymbolStore {
26 private:
27     std::array<std::string, MAX_TOKENS> symbols;
28     std::unordered_map<std::string, size_t> _multiDelimiters;
29     std::unordered_map<std::string, size_t> _keywords;
30     std::unordered_map<std::string, size_t> _identifiers;
31     std::unordered_map<std::string, size_t> _literals;
32
33     void declareAscii(char character);
34     void declareMultiDelimiter(const std::string& delimiter);
35     void declareKeyword(const std::string& keyword);
36
37 public:
38     SymbolStore();
39
40     size_t resolveMultiDelimiter(const std::string& delimiter);
41     size_t resolveKeyword(const std::string& keyword);
42     size_t resolveIdentifier(const std::string& identifier);
43     size_t resolveLiteral(const std::string& literal);
44
45     bool isKeyword(const std::string& token) const;
46     bool isMultiDelimiter(const std::string& token) const;
47
48     const std::string& lookup(size_t code) const;
49     SymbolType lookupType(size_t code) const;
50     std::vector<std::pair<std::string, size_t>> identifiers() const;
51     std::vector<std::pair<std::string, size_t>> literals() const;
52 };
```



```
1 #include "SymbolStore.hpp"
2
3 #include <algorithm>
4 #include <format>
5 #include <stdexcept>
6
7 SymbolStore::SymbolStore() {
8     // Initialize ASCII symbols
9     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
10         declareAscii(static_cast<char>(i));
11     }
12
13     // Initialize multi-character delimiters
14     declareMultiDelimiter(MultiDelimiter::Exp);
15
16     // Initialize keywords
17     declareKeyword(Keyword::Program);
18     declareKeyword(Keyword::Const);
19     declareKeyword(Keyword::Begin);
20     declareKeyword(Keyword::End);
21 }
22
23 void SymbolStore::declareAscii(const char character) {
24     symbols[static_cast<unsigned char>(character)] = std::string(1, character);
25 }
26
27 void SymbolStore::declareMultiDelimiter(const std::string& delimiter) {
28     if (_multiDelimiters.contains(delimiter)) {
29         throw std::invalid_argument(
30             std::format("Multi-delimiter '{}{}' is already declared", delimiter)
31         );
32     }
33
34     size_t code = MULTI_DELIMITERS_OFFSET + _multiDelimiters.size();
35
36     if (code >= KEYWORDS_OFFSET) {
37         throw std::overflow_error(
38             "Exceeded maximum number of multi-delimiters"
39         );
40     }
41
42     _multiDelimiters[delimiter] = code;
43     symbols[code] = delimiter;
44 }
45
46 size_t SymbolStore::resolveMultiDelimiter(const std::string& delimiter) {
47     if (!_multiDelimiters.contains(delimiter)) {
48         throw std::invalid_argument(
49             std::format("'{}{}' is not a multi-delimiter", delimiter)
50         );
51     }
52
53     return _multiDelimiters[delimiter];
54 }
55
56 void SymbolStore::declareKeyword(const std::string& keyword) {
57     if (_keywords.contains(keyword)) {
58         throw std::invalid_argument(
59             std::format("Keyword '{}{}' is already declared", keyword)
60         );
61     }
62 }
```

```

63     size_t code = KEYWORDS_OFFSET + _keywords.size();
64
65     if (code >= LITERALS_OFFSET) {
66         throw std::overflow_error("Exceeded maximum number of keywords");
67     }
68
69     _keywords[keyword] = code;
70     symbols[code] = keyword;
71 }
72
73 size_t SymbolStore::resolveKeyword(const std::string& keyword) {
74     if (!_keywords.contains(keyword)) {
75         throw std::invalid_argument(
76             std::format("{} is not a keyword", keyword)
77         );
78     }
79
80     return _keywords[keyword];
81 }
82
83 size_t SymbolStore::resolveIdentifier(const std::string& identifier) {
84     if (!_identifiers.contains(identifier)) {
85         size_t code = IDENTIFIERS_OFFSET + _identifiers.size();
86
87         if (code >= MAX_TOKENS) {
88             throw std::overflow_error("Exceeded maximum number of identifiers");
89         }
90
91         _identifiers[identifier] = code;
92         symbols[code] = identifier;
93     }
94
95     return _identifiers[identifier];
96 }
97
98 size_t SymbolStore::resolveLiteral(const std::string& literal) {
99     if (!_literals.contains(literal)) {
100         size_t code = LITERALS_OFFSET + _literals.size();
101
102         if (code >= IDENTIFIERS_OFFSET) {
103             throw std::overflow_error("Exceeded maximum number of literals");
104         }
105
106         _literals[literal] = code;
107         symbols[code] = literal;
108     }
109
110     return _literals[literal];
111 }
112
113 bool SymbolStore::isKeyword(const std::string& token) const {
114     return _keywords.contains(token);
115 }
116
117 bool SymbolStore::isMultiDelimiter(const std::string& token) const {
118     return _multiDelimiters.contains(token);
119 }
120
121 SymbolType SymbolStore::lookupType(size_t code) const {
122     if (code < MULTI_DELIMITERS_OFFSET) {
123         return SymbolType::Ascii;
124     }
125
126     if (code < KEYWORDS_OFFSET) {
127         return SymbolType::MultiDelimiter;
128     }
129

```

```

130     if (code < LITERALS_OFFSET) {
131         return SymbolType::Keyword;
132     }
133
134     if (code < IDENTIFIERS_OFFSET) {
135         return SymbolType::Literal;
136     }
137
138     if (code < MAX_TOKENS) {
139         return SymbolType::Identifier;
140     }
141
142     throw std::out_of_range(
143         std::format("Symbol code {} is out of range", code)
144     );
145 }
146
147 const std::string& SymbolStore::lookup(size_t code) const {
148     if (code >= symbols.size()) {
149         throw std::out_of_range(
150             std::format("Symbol code {} is out of range", code)
151         );
152     }
153
154     return symbols[code];
155 }
156
157 std::vector<std::pair<std::string, size_t>> SymbolStore::identifiers() const {
158     std::vector<std::pair<std::string, size_t>> result;
159
160     for (const auto& pair : _identifiers) {
161         result.emplace_back(pair.first, pair.second);
162     }
163
164     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
165         return a.second < b.second;
166     });
167
168     return result;
169 }
170
171 std::vector<std::pair<std::string, size_t>> SymbolStore::literals() const {
172     std::vector<std::pair<std::string, size_t>> result;
173
174     for (const auto& pair : _literals) {
175         result.emplace_back(pair.first, pair.second);
176     }
177
178     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
179         return a.second < b.second;
180     });
181
182     return result;
183 }

```

src/tokenizer/data/CharacterAttributes/CharacterAttributes.hpp

```
1 #pragma once
2
3 #include <array>
4
5 #include "symbols.constants.hpp"
6
7 enum class Attribute {
8     Whitespace,
9     Digit,
10    Letter,
11    Delimiter,
12    MultiDelimiter,
13    Comment,
14    Invalid,
15 };
16
17 class CharacterAttributes {
18 private:
19     std::array<Attribute, ASCII_TABLE_SIZE> attributes;
20
21 public:
22     CharacterAttributes();
23
24     [[nodiscard]] Attribute lookup(char character) const;
25 };
```

```
1 #include "CharacterAttributes.hpp"
2
3 CharacterAttributes::CharacterAttributes() {
4     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
5         attributes[i] = Attribute::Invalid;
6     }
7
8     // Whitespace
9     attributes[' '] = Attribute::Whitespace;
10    attributes['\t'] = Attribute::Whitespace;
11    attributes['\n'] = Attribute::Whitespace;
12    attributes['\v'] = Attribute::Whitespace;
13    attributes['\f'] = Attribute::Whitespace;
14    attributes['\r'] = Attribute::Whitespace;
15
16    // Digits
17    for (char ch = '0'; ch <= '9'; ch++) {
18        attributes[ch] = Attribute::Digit;
19    }
20
21    // Letters (A-Z)
22    for (char ch = 'A'; ch <= 'Z'; ch++) {
23        attributes[ch] = Attribute::Letter;
24    }
25
26    // Delimiters
27    attributes[';'] = Attribute::Delimiter;
28    attributes['.'] = Attribute::Delimiter;
29    attributes['='] = Attribute::Delimiter;
30    attributes['\'] = Attribute::Delimiter;
31    attributes[','] = Attribute::Delimiter;
32    attributes[')'] = Attribute::Delimiter;
33
34    // Multi-delimiters
35    attributes['$'] = Attribute::MultiDelimiter;
36
37    // Comment start
38    attributes['('] = Attribute::Comment;
39 }
40
41 Attribute CharacterAttributes::lookup(char character) const {
42     return attributes[static_cast<unsigned char>(character)];
43 }
```

```
1 #pragma once
2
3 #include <cstdint>
4
5 class Cursor {
6 private:
7     uint8_t tabSize = 4;
8
9     unsigned int _row = 0;
10    unsigned int _column = 0;
11
12 public:
13     Cursor();
14     Cursor(uint8_t tabSize);
15
16     void advance(char character);
17     [[nodiscard]] unsigned int row() const;
18     [[nodiscard]] unsigned int column() const;
19 };
```

src/tokenizer/data/Cursor/Cursor.cpp

```
1 #include "Cursor.hpp"
2
3 Cursor::Cursor() : Cursor(4) {}
4
5 Cursor::Cursor(uint8_t tabSize) : tabSize(tabSize) {};
6
7 void Cursor::advance(char character) {
8     switch (character) {
9         case '\r':
10             break;
11
12         case '\n':
13             _row++;
14             _column = 0;
15             break;
16
17         case '\t':
18             _column += tabSize - _column % tabSize;
19             break;
20
21         default:
22             _column++;
23             break;
24     }
25 }
26
27 unsigned int Cursor::row() const {
28     return _row;
29 }
30
31 unsigned int Cursor::column() const {
32     return _column;
33 }
```

src/tokenizer/data/FileSource/FileSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <fstream>
5 #include "Source.hpp"
6
7 class FileSource : public Source {
8 private:
9     std::ifstream file;
10     char character;
11
12 public:
13     FileSource(const std::string& filePath);
14
15     char current() override;
16     char read() override;
17     bool done() override;
18 };
```


src/tokenizer/data/FileSource/FileSource.cpp

```
1 #include "FileSource.hpp"
2
3 #include <stdexcept>
4
5 FileSource::FileSource(const std::string& filePath) :
6     file(filePath),
7     character(static_cast<char>(file.get())) {
8     if (!file.is_open()) {
9         throw std::runtime_error("Could not open file: " + filePath);
10    }
11 }
12
13 char FileSource::current() {
14     return character;
15 }
16
17 char FileSource::read() {
18     _cursor.advance(character);
19     character = static_cast<char>(file.get());
20
21     return character;
22 }
23
24 bool FileSource::done() {
25     return file.eof();
26 }
```

src/tokenizer/data/LexicalError/LexicalError.hpp

```
1 #pragma once
2
3 #include <cstdint>
4 #include <format>
5 #include <string>
6
7 struct LexicalError {
8     std::string message;
9     size_t row;
10    size_t column;
11
12    static auto InvalidMultiDelimiter(const std::string& token) {
13        return std::format("'{}' is not a valid multi-delimiter", token);
14    }
15
16    static auto UnclosedComment() {
17        return std::string("Comment not closed");
18    }
19
20    static auto InvalidCharacter(char character) {
21        return std::format("Invalid character '{}'", character);
22    }
23 };
```

src/tokenizer/data/Source/Source.hpp

```
1 #pragma once
2
3 #include <Cursor.hpp>
4
5 class Source {
6 protected:
7     Cursor _cursor;
8
9 public:
10     Source();
11     Source(Cursor cursor);
12
13     virtual ~Source() = default;
14
15     virtual char current() = 0;
16     virtual char read() = 0;
17     virtual bool done() = 0;
18
19     [[nodiscard]] const Cursor& cursor() const;
20 };
```

src/tokenizer/data/Source/Source.cpp

```
1 #include "Source.hpp"
2
3 Source::Source() = default;
4
5 Source::Source(Cursor cursor) : _cursor(cursor) {}
6
7 const Cursor& Source::cursor() const {
8     return _cursor;
9 }
```

src/tokenizer/data/StringSource/StringSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include "Source.hpp"
5
6 class StringSource : public Source {
7 private:
8     std::string source;
9     std::string::iterator iterator;
10
11 public:
12     StringSource(std::string source);
13
14     char current() override;
15     char read() override;
16     bool done() override;
17 };
```

src/tokenizer/data/StringSource/StringSource.cpp

```
1 #include "StringSource.hpp"
2
3 StringSource::StringSource(std::string source) :
4     source(std::move(source)),
5     iterator(this->source.begin()) {}
6
7 char StringSource::current() {
8     return *iterator;
9 }
10
11 char StringSource::read() {
12     _cursor.advance(*iterator);
13     iterator++;
14
15     return *iterator;
16 }
17
18 bool StringSource::done() {
19     return iterator == source.end();
20 }
```

src/tokenizer/data/Token/Token.hpp

```
1 #pragma once
2
3 #include <cstddef>
4
5 struct Token {
6     size_t code;
7     size_t row;
8     size_t column;
9 };
```

src/tokenizer/data/Tokenizer/Tokenizer.hpp

```
1 #pragma once
2
3 #include <vector>
4
5 #include "CharacterAttributes.hpp"
6 #include "LexicalError.hpp"
7 #include "Logger.hpp"
8 #include "Source.hpp"
9 #include "SymbolStore.hpp"
10 #include "Token.hpp"
11
12 class Tokenizer {
13 private:
14     Source& _source;
15     SymbolStore& _symbols;
16     CharacterAttributes& _attributes;
17     Logger<LexicalError>& _logger;
18
19     char _character;
20     std::string _token;
21     size_t _code;
22
23     std::vector<Token> _tokens;
24
25     void scanWhitespaces();
26     void scanInteger();
27     void scanString();
28     void scanComment();
29     void scanMultiDelimiter();
30     void scanDelimiter();
31     void scanInvalid();
32
33 public:
34     Tokenizer(
35         Source& source,
36         SymbolStore& symbols,
37         CharacterAttributes& attributes,
38         Logger<LexicalError>& logger
39     );
40
41     void scan();
42
43     void addToken(const Token& token);
44     void addError(const LexicalError& error);
45
46     [[nodiscard]] const std::vector<Token>& tokens() const;
47 };
```


src/tokenizer/data/Tokenizer/Tokenizer.cpp

```
1 #include "Tokenizer.hpp"
2
3 #include "CharacterAttributes.hpp"
4
5 // Automata state: START
6 Tokenizer::Tokenizer(
7     Source& source,
8     SymbolStore& symbols,
9     CharacterAttributes& attributes,
10    Logger<LexicalError>& logger
11 ) :
12     _source(source),
13     _symbols(symbols),
14     _attributes(attributes),
15     _logger(logger),
16     _character(source.current()) {}
17
18 void Tokenizer::scan() {
19     // Automata state: LOOP
20     while (!_source.done()) {
21         switch (_attributes.lookup(_character)) {
22             // Automata state: WHITESPACE
23             case Attribute::Whitespace:
24                 scanWhitespaces();
25                 break;
26
27             // Automata state: INTEGER
28             case Attribute::Digit:
29                 scanInteger();
30                 break;
31
32             // Automata state: STRING
33             case Attribute::Letter:
34                 scanString();
35                 break;
36
37             // Automata state: MULTI_DELIMITER
38             case Attribute::MultiDelimiter:
39                 scanMultiDelimiter();
40                 break;
41
42             // Automata state: COMMENT_START
43             case Attribute::Comment:
44                 scanComment();
45                 break;
46
47             // Automata state: DELIMITER
48             case Attribute::Delimiter:
49                 scanDelimiter();
50                 break;
51
52             // Automata state: ERROR
53             case Attribute::Invalid:
54                 scanInvalid();
55                 break;
56         }
57     }
58     // Automata state: END
59 }
60
61 void Tokenizer::scanWhitespaces() {
62     while (!_source.done() &&
```

```

63     _attributes.lookup(_character) == Attribute::Whitespace) {
64     _character = _source.read();
65 }
66 }
67
68 void Tokenizer::scanInteger() {
69     while (!_source.done() && _attributes.lookup(_character) == Attribute::Digit) {
70         _token += _character;
71         _character = _source.read();
72     };
73
74     // Automata state: WRITE
75     _code = _symbols.resolveLiteral(_token);
76
77     addToken({
78         .code = _code,
79         .row = _source.cursor().row(),
80         .column = _source.cursor().column() -
81             _token.length(), // Point to first character of the token
82     });
83
84     _token.clear();
85 }
86
87 void Tokenizer::scanString() {
88     while (!_source.done() &&
89         (_attributes.lookup(_character) == Attribute::Letter ||
90         _attributes.lookup(_character) == Attribute::Digit)) {
91         _token += _character;
92         _character = _source.read();
93     }
94
95     // Automata state: WRITE
96     if (_symbols.isKeyword(_token)) {
97         _code = _symbols.resolveKeyword(_token);
98     } else {
99         _code = _symbols.resolveIdentifier(_token);
100     }
101
102     addToken({
103         .code = _code,
104         .row = _source.cursor().row(),
105         .column = _source.cursor().column() -
106             _token.length(), // Point to first character of the token
107     });
108
109     _token.clear();
110 }
111
112 void Tokenizer::scanMultiDelimiter() {
113     size_t startRow = _source.cursor().row();
114     size_t startCol = _source.cursor().column();
115
116     _token += _character;
117     _character = _source.read();
118
119     if (_source.done() || _attributes.lookup(_character) != Attribute::Letter) {
120         addError({
121             .message = LexicalError::InvalidMultiDelimiter(_token),
122             .row = startRow,
123             .column = startCol,
124         });
125
126         _token.clear();
127
128         return;
129     }

```

```

130
131     while (!_source.done() &&
132            (_attributes.lookup(_character) == Attribute::Letter)) {
133         _token += _character;
134         _character = _source.read();
135     }
136
137     if (!_symbols.isMultiDelimiter(_token)) {
138         addError({
139             .message = LexicalError::InvalidMultiDelimiter(_token),
140             .row = startRow,
141             .column = startCol,
142         });
143
144         _token.clear();
145         return;
146     }
147
148     _code = _symbols.resolveMultiDelimiter(_token);
149
150     // Automata state: WRITE
151     addToken({
152         .code = _code,
153         .row = startRow,
154         .column = startCol,
155     });
156
157     _token.clear();
158 }
159
160 void Tokenizer::scanComment() {
161     size_t startRow = _source.cursor().row();
162     size_t startCol = _source.cursor().column();
163
164     _character = _source.read();
165
166     // Automata state: WRITE
167     if (_character != '*') {
168         addToken({
169             .code = static_cast<size_t>('('),
170             .row = startRow,
171             .column = startCol,
172         });
173
174         return;
175     }
176
177     _character = _source.read();
178
179     // Automata state: COMMENT
180     while (!_source.done()) {
181         // Automata state: COMMENT_CLOSE
182         if (_character == '*') {
183             _character = _source.read();
184
185             // Automata state: COMMENT_END
186             if (_character == ')') {
187                 _character = _source.read();
188
189                 return;
190             }
191         } else {
192             _character = _source.read();
193         }
194     }
195
196     // Automata state: ERROR

```

```

197     addError({
198         .message = LexicalError::UnclosedComment(),
199         .row = startRow,
200         .column = startCol,
201     });
202 }
203
204 void Tokenizer::scanDelimiter() {
205     addToken({
206         .code = static_cast<size_t>(_character),
207         .row = _source.cursor().row(),
208         .column = _source.cursor().column(),
209     });
210
211     _character = _source.read();
212 }
213
214 void Tokenizer::scanInvalid() {
215     addError({
216         .message = LexicalError::InvalidCharacter(_character),
217         .row = _source.cursor().row(),
218         .column = _source.cursor().column(),
219     });
220
221     _character = _source.read();
222 }
223
224 const std::vector<Token>& Tokenizer::tokens() const {
225     return _tokens;
226 }
227
228 void Tokenizer::addToken(const Token& token) {
229     _tokens.push_back(token);
230 }
231
232 void Tokenizer::addError(const LexicalError& error) {
233     _logger.message(error);
234 }

```

src/tokenizer/view/IdentifiersTableView/IdentifiersTableView.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <utility>
5 #include <vector>
6
7 #include <TableView.hpp>
8
9 using SymbolEntry = std::pair<std::string, size_t>;
10
11 class IdentifiersTableView {
12 private:
13     TableView<SymbolEntry> _table;
14
15 public:
16     IdentifiersTableView(
17         std::string title,
18         const std::vector<SymbolEntry>& data
19     );
20
21     void print() const;
22 };
```

```
1 #include "IdentifiersTableView.hpp"
2
3 IdentifiersTableView::IdentifiersTableView(
4     std::string title,
5     const std::vector<SymbolEntry>& data
6 ) :
7     _table(std::move(title), data) {
8     _table
9         .addColumn(
10             "Code", "{:>5}",
11             [](const auto& pair) { return std::to_string(pair.second); }
12         )
13         .addColumn(
14             "Value", "{:<30}",
15             [](const auto& pair) { return pair.first; }
16         );
17 }
18
19 void IdentifiersTableView::print() const {
20     _table.print();
21 }
```

src/tokenizer/view/LiteralsTableView/LiteralsTableView.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <utility>
5 #include <vector>
6
7 #include <TableView.hpp>
8
9 using SymbolEntry = std::pair<std::string, size_t>;
10
11 class LiteralsTableView {
12 private:
13     TableView<SymbolEntry> _table;
14
15 public:
16     LiteralsTableView(
17         std::string title,
18         const std::vector<SymbolEntry>& data
19     );
20
21     void print() const;
22 };
```

```
1 #include "LiteralsTableView.hpp"
2
3 LiteralsTableView::LiteralTableView(
4     std::string title,
5     const std::vector<SymbolEntry>& data
6 ) :
7     _table(std::move(title), data) {
8     _table
9         .addColumn(
10             "Code", "{:>5}",
11             [](const auto& pair) { return std::to_string(pair.second); }
12         )
13         .addColumn(
14             "Value", "{:<30}",
15             [](const auto& pair) { return pair.first; }
16         );
17 }
18
19 void LiteralsTableView::print() const {
20     _table.print();
21 }
```


src/tokenizer/view/TokensTableView/TokensTableView.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <vector>
5
6 #include <SymbolStore.hpp>
7 #include <TableView.hpp>
8 #include <Tokenizer.hpp>
9
10 class TokensTableView {
11 private:
12     TableView<Token> _table;
13
14 public:
15     TokensTableView(
16         std::string title,
17         const SymbolStore& symbols,
18         const std::vector<Token>& data
19     );
20
21     void print() const;
22 };
```

```
1  #include "TokensTableView.hpp"
2
3  std::string getTypeLabel(SymbolType type) {
4      switch (type) {
5          case SymbolType::Ascii:
6              return "Delimiter";
7          case SymbolType::MultiDelimiter:
8              return "Multi-Delimiter";
9          case SymbolType::Keyword:
10             return "Keyword";
11          case SymbolType::Literal:
12             return "Literal";
13          case SymbolType::Identifier:
14             return "Identifier";
15          default:
16             return "N/A";
17      }
18  }
19
20  TokensTableView::TokensTableView(
21      std::string title,
22      const SymbolStore& symbols,
23      const std::vector<Token>& data
24  ) :
25      _table(std::move(title), data) {
26      _table
27          .addColumn(
28              "Code", "{:>5}",
29              [](const auto& token) { return std::to_string(token.code); }
30          )
31          .addColumn(
32              "Row", "{:>3}",
33              [](const auto& token) { return std::to_string(token.row + 1); }
34          )
35          .addColumn(
36              "Col", "{:>3}",
37              [](const auto& token) { return std::to_string(token.column + 1); }
38          )
39          .addColumn(
40              "Type", "{:<15}",
41              [&symbols](const auto& token) {
42                  return getTypeLabel(symbols.lookupType(token.code));
43              }
44          )
45          .addColumn("Value", "{:<30}", [&symbols](const auto& token) {
46              return std::string(symbols.lookup(token.code));
47          });
48  }
49
50  void TokensTableView::print() const {
51      _table.print();
52  }
```

src/types/data/Type/Type.hpp

```
1 #pragma once
2
3 enum class Type {
4     ComplexInteger,
5     ComplexFloat,
6 };
```

src/ui/ui.constants.hpp

```
1 #pragma once
2
3 namespace Box {
4 constexpr auto Horizontal = "-";
5 constexpr auto Vertical = "|";
6 constexpr auto DownAndRight = "┐";
7 constexpr auto DownAndLeft = "└";
8 constexpr auto UpAndRight = "┌";
9 constexpr auto UpAndLeft = "┘";
10 constexpr auto VerticalAndRight = "┤";
11 constexpr auto VerticalAndLeft = "├";
12 constexpr auto DownAndHorizontal = "┴";
13 constexpr auto UpAndHorizontal = "┬";
14 constexpr auto VerticalAndHorizontal = "┼";
15 } // namespace Box
```

src/ui/TableView/TableView.hpp

```
1  #pragma once
2
3  #include <format>
4  #include <functional>
5  #include <iostream>
6  #include <string>
7  #include <vector>
8
9  #include <ui.constants.hpp>
10
11 template <typename T>
12 class TableView {
13 private:
14     struct Column {
15         std::string name;
16         std::string format;
17         std::function<std::string(const T&)> extractor;
18     };
19
20     std::string _title;
21     std::vector<T> _data;
22     std::vector<Column> _columns;
23     std::ostream& _out;
24
25     [[nodiscard]] std::string border(
26         const std::string& left,
27         const std::string& junction,
28         const std::string& right
29     ) const;
30     [[nodiscard]] std::string header() const;
31     [[nodiscard]] std::string row(const T& item) const;
32     [[nodiscard]] size_t columnWidth(const Column& col) const;
33     static size_t parseWidth(const std::string& fmt);
34
35 public:
36     TableView(
37         std::string title,
38         std::vector<T> data,
39         std::ostream& out = std::cout
40     );
41
42     TableView& addColumn(
43         const std::string& colName,
44         const std::string& format,
45         std::function<std::string(const T&)> extractor
46     );
47
48     void print() const;
49 };
50
51 #include "TableView.tpp"
```

```
1  template <typename T>
2  TableView<T>::TableView(
3      std::string title,
4      std::vector<T> data,
5      std::ostream& out
6  ) :
7      _title(std::move(title)),
8      _data(std::move(data)),
9      _out(out) {}
10
11 template <typename T>
12 TableView<T>& TableView<T>::addColumn(
13     const std::string& colName,
14     const std::string& format,
15     std::function<std::string(const T&)> extractor
16 ) {
17     _columns.push_back({colName, format, std::move(extractor)});
18
19     return *this;
20 }
21
22 template <typename T>
23 void TableView<T>::print() const {
24     if (!_title.empty()) {
25         _out << _title << "\n";
26     }
27
28     _out << border(Box::DownAndRight, Box::DownAndHorizontal, Box::DownAndLeft) << "\n";
29     _out << header() << "\n";
30     _out << border(Box::VerticalAndRight, Box::VerticalAndHorizontal, Box::VerticalAndLeft) << "\n";
31
32     for (const auto& item : _data) {
33         _out << row(item) << "\n";
34     }
35
36     _out << border(Box::UpAndRight, Box::UpAndHorizontal, Box::UpAndLeft) << "\n";
37 }
38
39 template <typename T>
40 std::string TableView<T>::border(
41     const std::string& left,
42     const std::string& junction,
43     const std::string& right
44 ) const {
45     std::string result = left;
46
47     for (size_t i = 0; i < _columns.size(); ++i) {
48         size_t width = columnWidth(_columns[i]);
49
50         for (size_t j = 0; j < width + 2; ++j) {
51             result += Box::Horizontal;
52         }
53
54         result += (i < _columns.size() - 1) ? junction : right;
55     }
56
57     return result;
58 }
59
60 template <typename T>
61 std::string TableView<T>::header() const {
62     std::string result;
```

```

63     result += Box::Vertical;
64
65     for (const auto& column : _columns) {
66         result += std::format(" {:<{}} ", column.name, columnWidth(column));
67         result += Box::Vertical;
68     }
69
70     return result;
71 }
72
73 template <typename T>
74 std::string TableView<T>::row(const T& item) const {
75     std::string result;
76     result += Box::Vertical;
77
78     for (const auto& column : _columns) {
79         std::string value = column.extractor(item);
80         result += " " +
81             std::vformat(column.format, std::make_format_args(value)) +
82             " ";
83         result += Box::Vertical;
84     }
85
86     return result;
87 }
88
89 template <typename T>
90 size_t TableView<T>::columnWidth(const Column& column) const {
91     return std::max(parseWidth(column.format), column.name.length());
92 }
93
94 template <typename T>
95 size_t TableView<T>::parseWidth(const std::string& format) {
96     size_t width = 0;
97
98     for (char character : format) {
99         if (character >= '0' && character <= '9') {
100             width = (width * 10) + (character - '0');
101         }
102     }
103
104     return width;
105 }

```

src/ui/TreeView/TreeView.hpp

```
1  #pragma once
2
3  #include <functional>
4  #include <iostream>
5  #include <string>
6
7  #include <ui.constants.hpp>
8  #include <Tree.hpp>
9
10 template <typename T>
11 class TreeView {
12 private:
13     std::string _title;
14     const Tree<T>& _tree;
15     std::ostream& _out;
16     std::function<std::string(const T&)> _nodeFormatter;
17     std::function<std::string(const T&)> _edgeFormatter;
18
19     void printNode(
20         const TreeNode<T>& node,
21         const std::string& prefix,
22         bool isLast
23     ) const;
24
25 public:
26     TreeView(std::string title, const Tree<T>& tree, std::ostream& out = std::cout);
27
28     TreeView& setNodeFormatter(std::function<std::string(const T&)> formatter);
29     TreeView& setEdgeFormatter(std::function<std::string(const T&)> formatter);
30
31     void print() const;
32 };
33
34 #include "TreeView.hpp"
```



```
1  template <typename T>
2  TreeView<T>::TreeView(std::string title, const Tree<T>& tree, std::ostream& out) :
3      _title(std::move(title)),
4      _tree(tree),
5      _out(out),
6      _nodeFormatter(nullptr),
7      _edgeFormatter(nullptr) {}
8
9  template <typename T>
10 TreeView<T>& TreeView<T>::setNodeFormatter(
11     std::function<std::string(const T&)> formatter
12 ) {
13     _nodeFormatter = std::move(formatter);
14
15     return *this;
16 }
17
18 template <typename T>
19 TreeView<T>& TreeView<T>::setEdgeFormatter(
20     std::function<std::string(const T&)> formatter
21 ) {
22     _edgeFormatter = std::move(formatter);
23
24     return *this;
25 }
26
27 template <typename T>
28 void TreeView<T>::print() const {
29     auto& root = _tree.root();
30
31     _out << _title << "\n";
32     _out << _nodeFormatter(root.data()) << "\n";
33
34     for (const auto& child : root.children()) {
35         printNode(*child, "", child == root.children().back());
36     }
37 }
38
39 template <typename T>
40 void TreeView<T>::printNode(
41     const TreeNode<T>& node,
42     const std::string& prefix,
43     bool isLast
44 ) const {
45     std::string source = isLast
46         ? Box::UpAndRight
47         : Box::VerticalAndRight;
48
49     std::string horizontalConnector = Box::Horizontal;
50
51     std::string edgeLabel = _edgeFormatter(node.data());
52
53     std::string nodeLabel = _nodeFormatter(node.data());
54
55     std::string verticalConnector = isLast
56         ? " "
57         : Box::Vertical;
58
59     _out << prefix << source << horizontalConnector;
60
61     _out << edgeLabel << horizontalConnector;
62 }
```

```
63     _out << nodeLabel << "\n";
64
65     std::string childPrefix = prefix + verticalConnector + std::string(edgeLabel.size() + 2, ' ');
66
67     for (const auto& child : node.children()) {
68         printNode(*child, childPrefix, child == node.children().back());
69     }
70 }
```

src/utility/Args/Args.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <unordered_map>
5 #include <vector>
6
7 class Args {
8 private:
9     struct StringArg {
10         std::string shortName;
11         bool required;
12         std::string value;
13         bool provided = false;
14     };
15
16     struct FlagArg {
17         std::string shortName;
18         bool value = false;
19     };
20
21     std::string _program;
22     std::vector<std::string> _argv;
23
24     std::unordered_map<std::string, StringArg> _strings;
25     std::unordered_map<std::string, FlagArg> _flags;
26     std::unordered_map<std::string, std::string> _aliases;
27
28     void printUsage() const;
29
30 public:
31     Args(int argc, char* argv[]);
32
33     Args& expectString(
34         const std::string& name,
35         const std::string& shortName,
36         bool required = false
37     );
38     Args& expectFlag(
39         const std::string& name,
40         const std::string& shortName
41     );
42
43     Args& parse();
44
45     [[nodiscard]] const std::string& getString(const std::string& name) const;
46     [[nodiscard]] bool getFlag(const std::string& name) const;
47 };
```

src/utility/Args/Args.cpp

```
1 #include "Args.hpp"
2
3 #include <format>
4 #include <iostream>
5 #include <stdexcept>
6
7 Args::Args(int argc, char* argv[]) : _program(argv[0]) {
8     for (int i = 1; i < argc; i++) {
9         _argv.emplace_back(argv[i]);
10    }
11 }
12
13 Args& Args::expectString(
14     const std::string& name,
15     const std::string& shortName,
16     bool required
17 ) {
18     _strings[name] = {shortName, required, "", false};
19     _aliases["--" + name] = name;
20     _aliases["-" + shortName] = name;
21     return *this;
22 }
23
24 Args& Args::expectFlag(
25     const std::string& name,
26     const std::string& shortName
27 ) {
28     _flags[name] = {shortName, false};
29     _aliases["--" + name] = name;
30     _aliases["-" + shortName] = name;
31     return *this;
32 }
33
34 Args& Args::parse() {
35     for (size_t i = 0; i < _argv.size(); i++) {
36         const auto& arg = _argv[i];
37
38         auto alias = _aliases.find(arg);
39
40         if (alias == _aliases.end()) {
41             std::cerr << "Unknown argument: " << arg << "\n";
42             printUsage();
43             exit(1);
44         }
45
46         const auto& name = alias->second;
47
48         auto stringArg = _strings.find(name);
49         if (stringArg != _strings.end()) {
50             if (i + 1 >= _argv.size()) {
51                 std::cerr << "Missing value for: " << arg << "\n";
52                 printUsage();
53                 exit(1);
54             }
55
56             stringArg->second.value = _argv[++i];
57             stringArg->second.provided = true;
58             continue;
59         }
60
61         auto flagArg = _flags.find(name);
62         if (flagArg != _flags.end()) {
```

```

63         flagArg->second.value = true;
64     }
65 }
66
67 for (const auto& [name, arg] : _strings) {
68     if (arg.required && !arg.provided) {
69         std::cerr << "Missing required argument: --" << name << "\n";
70         printUsage();
71         exit(1);
72     }
73 }
74
75 return *this;
76 }
77
78 void Args::printUsage() const {
79     std::cerr << "\nUsage: " << _program;
80
81     for (const auto& [name, arg] : _strings) {
82         if (arg.required) {
83             std::cerr << " --" << name << " <value>";
84         }
85     }
86
87     std::cerr << " [options]\n\nOptions:\n";
88
89     for (const auto& [name, arg] : _strings) {
90         std::cerr << std::format(
91             "  --{}, -{} <value>{}\n", name, arg.shortName,
92             arg.required ? " (required)" : ""
93         );
94     }
95
96     for (const auto& [name, arg] : _flags) {
97         std::cerr << std::format("  --{}, -{}\n", name, arg.shortName);
98     }
99 }
100
101 const std::string& Args::getString(const std::string& name) const {
102     auto it = _strings.find(name);
103
104     if (it == _strings.end()) {
105         throw std::runtime_error("Unknown string argument: " + name);
106     }
107
108     return it->second.value;
109 }
110
111 bool Args::getFlag(const std::string& name) const {
112     auto it = _flags.find(name);
113
114     if (it == _flags.end()) {
115         throw std::runtime_error("Unknown flag argument: " + name);
116     }
117
118     return it->second.value;
119 }

```

src/utility/Logger/Logger.hpp

```
1 #pragma once
2
3 #include <functional>
4 #include <iostream>
5 #include <string>
6 #include <vector>
7
8 template <typename T>
9 class Logger {
10 private:
11     std::string _name;
12     std::function<std::string(const T&)> _formatter;
13     std::ostream& _out;
14     std::vector<T> _messages;
15
16 public:
17     Logger(
18         std::string name,
19         std::function<std::string(const T&)> formatter,
20         std::ostream& out = std::cout
21     );
22
23     void message(const T& data);
24
25     [[nodiscard]] const std::vector<T>& messages() const;
26     void clear();
27 };
28
29 #include "Logger.hpp"
```

```
1  template <typename T>
2  Logger<T>::Logger(
3      std::string name,
4      std::function<std::string(const T&)> formatter,
5      std::ostream& out
6  ) :
7      _name(std::move(name)), _formatter(std::move(formatter)), _out(out) {}
8
9  template <typename T>
10 void Logger<T>::message(const T& data) {
11     _messages.push_back(data);
12     _out << _name << " | " << _formatter(data) << "\n";
13 }
14
15 template <typename T>
16 const std::vector<T>& Logger<T>::messages() const {
17     return _messages;
18 }
19
20 template <typename T>
21 void Logger<T>::clear() {
22     _messages.clear();
23 }
```

```
1 #pragma once
2
3 #include <deque>
4 #include <functional>
5 #include <optional>
6 #include <vector>
7
8 template <typename T>
9 class Queue {
10 private:
11     std::deque<T> _items;
12
13 public:
14     Queue();
15     Queue(const std::vector<T>& items);
16
17     void push(const T& item);
18
19     std::optional<T> pop();
20     std::optional<T> popUntil(const std::function<bool(const T&)>& predicate);
21
22     void clear();
23
24     std::optional<T> peek() const;
25     bool isEmpty() const;
26     size_t size() const;
27 };
28
29 #include "Queue.hpp"
```



```

1  template <typename T>
2  Queue<T>::Queue() : _items() {}
3
4  template <typename T>
5  Queue<T>::Queue(const std::vector<T>& items) :
6      _items(items.begin(), items.end()) {}
7
8  template <typename T>
9  void Queue<T>::push(const T& item) {
10     _items.push_back(item);
11 }
12
13 template <typename T>
14 std::optional<T> Queue<T>::pop() {
15     if (_items.empty()) {
16         return std::nullopt;
17     }
18
19     T item = std::move(_items.front());
20     _items.pop_front();
21
22     return item;
23 }
24
25 template <typename T>
26 std::optional<T> Queue<T>::popUntil(
27     const std::function<bool(const T&)>& predicate
28 ) {
29     while (!_items.empty()) {
30         if (predicate(_items.front())) {
31             return _items.front();
32         }
33
34         _items.pop_front();
35     }
36
37     return std::nullopt;
38 }
39
40 template <typename T>
41 void Queue<T>::clear() {
42     _items.clear();
43 }
44
45 template <typename T>
46 std::optional<T> Queue<T>::peek() const {
47     if (_items.empty()) {
48         return std::nullopt;
49     }
50
51     return _items.front();
52 }
53
54 template <typename T>
55 bool Queue<T>::isEmpty() const {
56     return _items.empty();
57 }
58
59 template <typename T>
60 size_t Queue<T>::size() const {
61     return _items.size();
62 }

```

src/utility/Stack/Stack.hpp

```
1 #pragma once
2
3 #include <vector>
4 #include <optional>
5 #include <functional>
6
7 template <typename T>
8 class Stack {
9 private:
10     std::vector<T> _items;
11
12 public:
13     Stack();
14     Stack(const std::vector<T>& items);
15
16     void push(const T& item);
17
18     std::optional<T> pop();
19     std::optional<T> popUntil(const std::function<bool(const T&)>& predicate);
20
21     void clear();
22
23     std::optional<T> peek() const;
24     bool isEmpty() const;
25     size_t size() const;
26 };
27
28 #include "Stack.cpp"
```

```
1  template <typename T>
2  Stack<T>::Stack() : _items() {}
3
4  template <typename T>
5  Stack<T>::Stack(const std::vector<T>& items) : _items(items) {}
6
7  template <typename T>
8  void Stack<T>::push(const T& item) {
9      _items.push_back(item);
10 }
11
12 template <typename T>
13 std::optional<T> Stack<T>::pop() {
14     if (_items.empty()) {
15         return std::nullopt;
16     }
17
18     T item = std::move(_items.back());
19     _items.pop_back();
20
21     return item;
22 }
23
24 template <typename T>
25 std::optional<T> Stack<T>::popUntil(const std::function<bool(const T&)>& predicate) {
26     while (!_items.empty()) {
27         if (predicate(_items.back())) {
28             return _items.back();
29         }
30
31         _items.pop_back();
32     }
33
34     return std::nullopt;
35 }
36
37 template <typename T>
38 void Stack<T>::clear() {
39     _items.clear();
40 }
41
42 template <typename T>
43 std::optional<T> Stack<T>::peek() const {
44     if (_items.empty()) {
45         return std::nullopt;
46     }
47
48     return _items.back();
49 }
50
51 template <typename T>
52 bool Stack<T>::isEmpty() const {
53     return _items.empty();
54 }
55
56 template <typename T>
57 size_t Stack<T>::size() const {
58     return _items.size();
59 }
```

src/utility/Tree/Tree.hpp

```
1 #pragma once
2
3 #include <TreeNode.hpp>
4
5 template <typename T>
6 class Tree {
7 private:
8     std::unique_ptr<TreeNode<T>> _root;
9
10 public:
11     Tree(T data);
12
13     TreeNode<T>& root();
14     const TreeNode<T>& root() const;
15 };
16
17 #include "Tree.cpp"
```

```
1  template <typename T>
2  Tree<T>::Tree(T data) : _root(std::make_unique<TreeNode<T>>(data)) {}
3
4  template <typename T>
5  TreeNode<T>& Tree<T>::root() {
6      return *_root;
7  }
8
9  template <typename T>
10 const TreeNode<T>& Tree<T>::root() const {
11     return *_root;
12 }
```

src/utility/Tree/Node/TreeNode.hpp

```
1 #pragma once
2
3 #include <vector>
4 #include <memory>
5
6 template <typename T>
7 class TreeNode {
8 private:
9     T _data;
10    std::vector<std::unique_ptr<TreeNode<T>>> _children;
11
12 public:
13     TreeNode(T data);
14     TreeNode(const TreeNode&) = delete;
15     TreeNode& operator=(const TreeNode&) = delete;
16     TreeNode(TreeNode&&) = default;
17     TreeNode& operator=(TreeNode&&) = default;
18
19     const T& data() const;
20     T& data();
21
22     const std::vector<std::unique_ptr<TreeNode<T>>>& children() const;
23     TreeNode<T>& grow(T data);
24     void prune();
25 };
26
27 #include "TreeNode.cpp"
```

```
1  template <typename T>
2  TreeNode<T>::TreeNode(T data) : _data(data) {}
3
4  template <typename T>
5  const T& TreeNode<T>::data() const {
6      return _data;
7  }
8
9  template <typename T>
10 T& TreeNode<T>::data() {
11     return _data;
12 }
13
14 template <typename T>
15 const std::vector<std::unique_ptr<TreeNode<T>>>& TreeNode<T>::children() const {
16     return _children;
17 }
18
19 template <typename T>
20 TreeNode<T>& TreeNode<T>::grow(T data) {
21     _children.push_back(std::make_unique<TreeNode<T>>(data));
22     return *_children.back();
23 }
24
25 template <typename T>
26 void TreeNode<T>::prune() {
27     _children.clear();
28 }
```